REPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Carthage
Institut National des Sciences Appliquées et de Technologie

Rapport de Travaux Pratiques

Module : DevOps

Filière : Réseaux Informatiques et Télécommunications (RT5) – Groupe 1

TP2 : Kubernetes – Principes de base

Partie A : Réalisation et compréhension des vidéos

Réalisé par

Hanine BEN AMOR
Farah AYEB
Emna DRIHEM
Yasmine FERJANI

Encadré par : **Mme Saloua BEN YAHIA**

Année Universitaire : 2026 / 2027

Table des matières

Introduction générale **1**

2 Présentation de Kubernetes **2**

3 Installation de l'environnement **4**

3.1 Logiciels nécessaires et notre environnement 4

3.2 Installation de kubectl 5

3.3 Installation de Minikube 6

3.4 Relation entre Minikube et Docker 6

4 Le cluster Kubernetes **7**

4.1 Création du cluster avec Minikube 7

4.2 Exploration du nœud Kubernetes 8

5 Création d'un pod **13**

5.1 Création d'un pod unique 13

5.2 Exploration du pod 16

5.3 Création d'un alias pour kubectl 19

6 Création d'un déploiement **20**

6.1 Création et exploration d'un déploiement 20

6.2 Mise à l'échelle (scaling) du déploiement 22

6.3 Connexion à un pod via son adresse IP 24

7 Création d'un service **25**

7.1 Qu'est-ce qu'un Service ? 25

7.2 Création et exploration d'un service ClusterIP 26

7.3 Connexion au déploiement via le service ClusterIP 27

8 Suppression du déploiement et du service 29

8.1 Suppression 29

9 Exécution d'une application Node.js avec Kubernetes 30

9.1 Création de l'application web Node.js 30

9.2 Dockerisation de l'application 31

9.3 Publication de l'image sur Docker Hub 32

9.4 Création d'un déploiement à partir de l'image personnalisée 33

9.5 Mise à l'échelle du déploiement de l'image personnalisée 34

9.6 Création d'un service NodePort 36

9.7 Création d'un service LoadBalancer 39

10 Mises à jour progressives (rolling updates) et récupération des pods 41

10.1 Mise à jour progressive du déploiement 41

10.2 Que se passe-t-il quand un pod est supprimé ? 47

11 Tableau de bord Kubernetes (Dashboard) 48

12 Fichiers de spécification YAML 53

12.1 Approche impérative vs déclarative 53

12.2 Documentation de l'API Kubernetes 56

12.3 Application du fichier de déploiement 56

12.4 Fichier de service 57

13 Création de plusieurs déploiements 60

13.1 Suppression via les fichiers YAML et architecture cible 60

13.2 Application `k8s-web-to-nginx` 61

13.3 Construction et publication de l'image 63

13.4 Fichier combiné service + déploiement 64

13.5 Déploiement nginx 65

13.6 Application des deux fichiers 66

14 Interaction entre les déploiements 68

14.1 Test de la communication 68
14.2 Résolution DNS du nom du service 70
14.3 Suppression 72

15 Le runtime de conteneurs CRI-O **73**
15.1 Changement de runtime 73
15.2 Déploiement des applications sur CRI-O 75
15.3 Vérification 77

Conclusion générale **79**

Introduction générale

Cette partie suit le cours vidéo *Kubernetes for Beginners* : chaque vidéo a été reproduite sur notre machine, en adaptant les commandes à notre environnement (chapitre 3).

2

Présentation de Kubernetes

— **Kubernetes (K8s)** : plateforme open source d'**orchestration de conteneurs**. On décrit un **état désiré** (par exemple « 3 copies de cette image ») et Kubernetes l'applique en permanence : placement des conteneurs, redémarrage en cas de panne, mise à l'échelle et mises à jour sans interruption.

— **Pod** : la plus petite unité gérée par Kubernetes. Il contient un ou plusieurs conteneurs qui partagent la même adresse IP et peuvent partager des volumes. Un pod est éphémère : recréé, il obtient une nouvelle IP.

— **Cluster et nœuds** : un cluster est un ensemble de machines (**nœuds**). Le **control plane** pilote le cluster (API server, etcd, scheduler, controller manager) ; les **workers** exécutent les pods (kubelet, kube-proxy, runtime de conteneurs). Avec Minikube, un seul nœud joue les deux rôles.

— **Service** : donne une adresse IP et un nom DNS **stables** à un groupe de pods et répartit le trafic entre eux. Types principaux : ClusterIP (interne), NodePort (port sur chaque nœud), LoadBalancer (IP externe fournie par un cloud) ; voir chapitre 7.

— **kubectl** : l'outil en ligne de commande qui envoie des requêtes HTTPS à l'API server pour gérer les objets du cluster. Il lit l'adresse du cluster et les certificats dans `~/.kube/config`.

Hiérarchie des éléments (du plus grand au plus petit) :

```
Cluster
 +-- Nœuds (1 ou plusieurs machines : physiques, virtuelles, ou un conteneur avec
    ↪ Minikube)
      +-- Pods (0, 1 ou plusieurs pods par nœud)
           +-- Conteneurs (1 ou plusieurs par pod, le plus souvent 1)
```

— Un **cluster** contient un ou plusieurs **nœuds**. Notre cluster Minikube n'en contient qu'un : `minikube`.

— Un **nœud** exécute un ou plusieurs **pods** ; c'est le scheduler qui choisit sur quel nœud placer chaque pod.

— Un **pod** contient un ou plusieurs **conteneurs**. Ces conteneurs sont toujours placés **ensemble sur le même nœud**, partagent la **même adresse IP** et communiquent via `localhost`. Un pod ne s'étend jamais sur plusieurs nœuds.

— Un **conteneur** est une instance d'une **image** (par exemple `nginx` ou `hanine169/k8s-web-hello`) ; c'est lui qui exécute réellement l'application.

— Les objets de plus haut niveau s'appuient sur les pods : un **Deployment** gère un **ReplicaSet**, qui maintient un nombre donné de pods identiques ; un **Service** regroupe ces pods derrière une adresse stable.

3

Installation de l'environnement

3.1 Logiciels nécessaires et notre environnement

Le cours demande trois outils : un hyperviseur ou Docker (pour héberger le nœud), **kubectl** et **Minikube**. Le formateur travaille sous macOS avec VirtualBox. Notre environnement est différent (voir Tableau 3.1) :

TABLEAU 3.1 – Comparaison entre l'environnement du tutoriel et mon environnement

Élément	**Tutoriel**	**Notre environnement**
Système hôte	macOS	Windows 11 + **WSL 2** (Ubuntu 24.04.4 LTS, noyau `6.18.33.2-microsoft-standard-WSL2`)
Docker	—	**Docker Desktop** (version 29.6.1), avec l'intégration WSL activée
Driver Minikube	VirtualBox	**Docker** (`minikube start --driver=docker`)
Runtime de conteneurs dans le nœud	Docker Engine	**containerd** 2.3.4
Commande pour lister les conteneurs dans le nœud	`docker ps`	`sudo crictl ps`
Accès au nœud	`ssh docker@<ip>`	`minikube ssh`

Ces différences expliquent les quelques adaptations de commandes décrites dans la suite du rapport.

3.2 Installation de kubectl

kubectl a été installé dans WSL à partir du binaire officiel :

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt
    ↪ )/bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl
kubectl version --client
```

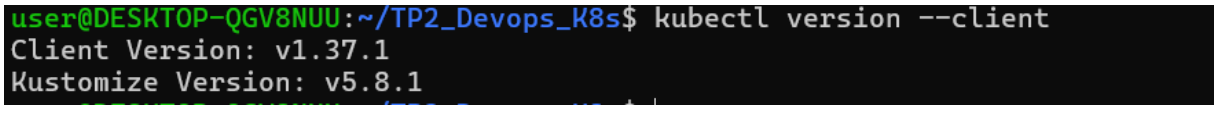


FIGURE 3.1 – `kubectl version --client` après l'installation : `Client Version: v1.37.1` et `Kustomize Version: v5.8.1`. kubectl est installé et utilisable. (À ce stade, aucun cluster n'est encore configuré : seule la version du client est demandée.)

3.3 Installation de Minikube

Minikube a été installé de la même façon, à partir du binaire Linux officiel :

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
```

Version installée : **Minikube v1.39.0**, qui déploie Kubernetes **v1.37.0**. La version apparaît dans la première ligne de `minikube start` (Figure 4.1, section 4.1).

Node.js et npm ne sont pas indispensables : l'application du chapitre 9 s'exécute dans une image Docker `node:alpine`, qui contient déjà Node.js. npm (installé côté Windows et accessible depuis WSL) a seulement servi à gérer les dépendances dans `package.json`.

3.4 Relation entre Minikube et Docker

Avec le driver `docker`, Minikube ne crée pas de machine virtuelle. Il crée **un conteneur Docker** nommé `minikube` sur l'hôte (WSL) et ce conteneur **joue le rôle du nœud Kubernetes**. À l'intérieur de ce nœud, un second runtime (containerd dans notre cas) exécute les conteneurs des pods. Il y a donc deux niveaux :

1. Docker (sur WSL) → exécute le conteneur `minikube` (= le nœud) ;
2. containerd (dans le nœud) → exécute les conteneurs des pods (nginx, coredns, etcd...).

C'est pourquoi `docker ps` sur l'hôte montre le conteneur `minikube`, mais pas les conteneurs des pods : pour les voir, il faut utiliser `crictl` à l'intérieur du nœud (section 4.2).

4

Le cluster Kubernetes

4.1 Création du cluster avec Minikube

```
minikube start --driver=docker --memory=2500 --cpus=2
minikube status
kubectl get nodes -o wide
```

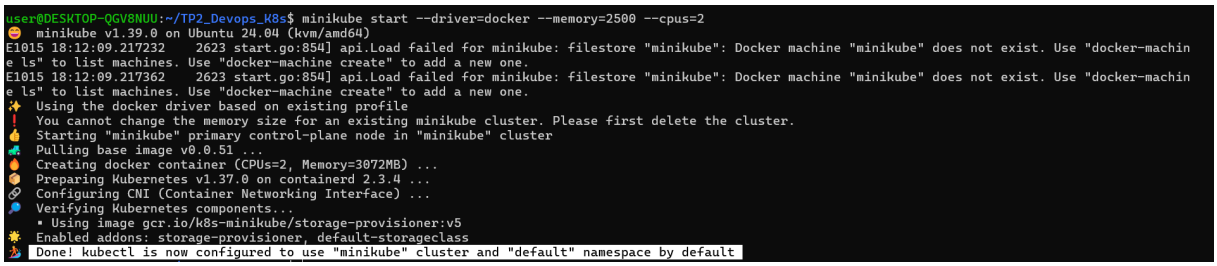


FIGURE 4.1 – `minikube start --driver=docker --memory=2500 --cpus=2` : Minikube v1.39.0 crée le conteneur Docker qui sert de nœud (`CPUs=2, Memory=3072MB`), prépare **Kubernetes v1.37.0 sur containerd 2.3.4**, configure le réseau (CNI), active les addons `storage-provisioner` et `default-storageclass`, puis configure `kubectl` pour utiliser le cluster `minikube`.

> Remarque sur les messages affichés : un profil `minikube` existait déjà d'un essai précédent, mais son conteneur Docker avait été supprimé. D'où les deux erreurs `api.Load failed ...` `Docker machine "minikube" does not exist` (Minikube recrée simplement le conteneur)

> et l'avertissement « You cannot change the memory size for an existing minikube cluster » : l'option `--memory=2500` est ignorée et c'est la mémoire du profil existant (3072 Mo) qui est utilisée. Pour changer la mémoire, il faudrait d'abord `minikube delete`.

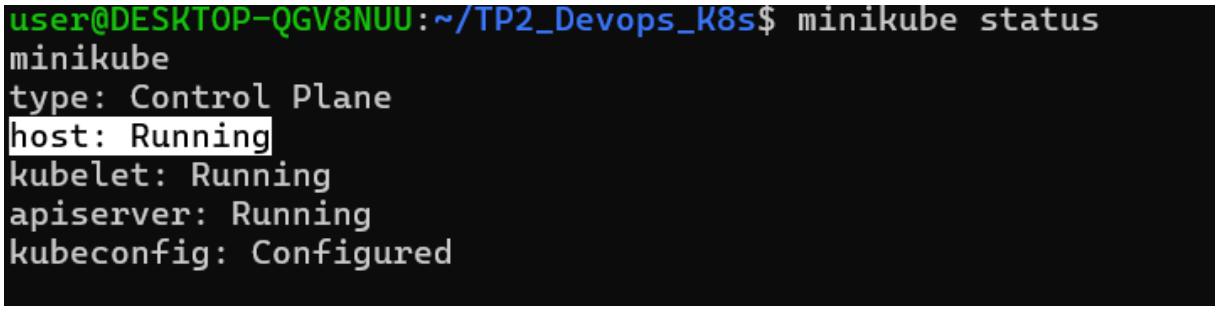


FIGURE 4.2 – `minikube status` : le nœud `minikube` est de type `Control Plane`, et le `host`, le `kubelet` et l'`apiserver` sont `Running` ; `kubeconfig: Configured` indique que `kubectl` est configuré pour ce cluster.



FIGURE 4.3 – `kubectl get nodes -o wide` : un seul nœud, `minikube`, `Ready`, rôle `control-plane`, Kubernetes v1.37.0, IP interne `192.168.49.2`, système `Debian GNU/Linux 12 (bookworm)` sur le noyau `6.18.33.2-microsoft-standard-WSL2`, et runtime **`containerd://2.3.4`**.

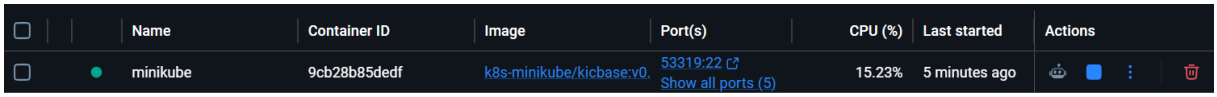


FIGURE 4.4 – Dans Docker Desktop, le nœud Kubernetes apparaît comme **un simple conteneur Docker** nommé `minikube` (image `kicbase`), ce qui illustre la relation entre Minikube et Docker décrite en 3.4.

4.2 Exploration du nœud Kubernetes

Récupération de l'adresse IP du nœud Minikube :

```
minikube ip
```

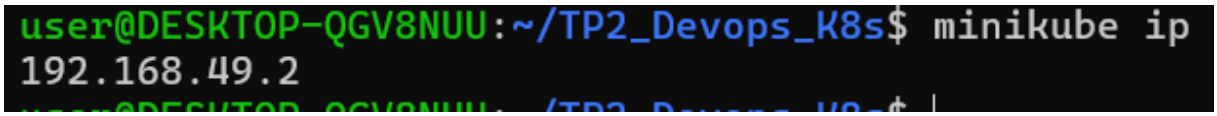


FIGURE 4.5 – `minikube ip` : le nœud Minikube possède l'adresse IP `192.168.49.2` sur le réseau Docker créé par Minikube.

Dans la vidéo, le formateur se connecte au nœud avec `ssh docker@<ip>`. Avec le driver Docker, cette commande ne fonctionne pas ; on utilise :

```
minikube ssh
```

Minikube ouvre automatiquement une session shell dans le nœud, sans avoir à gérer l'utilisateur, le mot de passe ou la clé SSH.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ minikube ssh
Linux minikube 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 U
TC 2026 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
docker@minikube:~$
```

FIGURE 4.6 – `minikube ssh` : la bannière indique un système Debian sur le noyau `6.18.33.2-microsoft-standard-WSL2` (le nœud partage le noyau de WSL), et l'invite devient `docker@minikube:~$` : on est à l'intérieur du nœud Kubernetes.

À l'intérieur du nœud, le formateur utilise `docker ps`. Le runtime de notre nœud étant containerd, on a utilisé l'outil `crictl` (client standard de l'interface CRI de Kubernetes) :

```
sudo crictl ps
```

```
docker@minikube:~$ sudo crictl ps
CONTAINER           IMAGE               CREATED             STATE               NAME                      ATTEMPT             POD ID              POD
                    NAMESPACE
37a171c537b22       520212b8b0fcd       14 minutes ago      Running             coredns                   0                   312bb0e074ae7       coredns-55
9f6c778d-hltn2
14ef4d7845902       6e38f40d628db       14 minutes ago      Running             storage-provisioner       0                   62be0467ffaa6       storage-pr
ovisioner
a9cd5401166d1       4626fe10df5b9       14 minutes ago      Running             kindnet-cni               0                   0563483c4e34b       kindnet-5q
hwr
99e156c67b004       d6a28daf3e6b0       15 minutes ago      Running             kube-proxy                0                   41ae595ba8853       kube-proxy
-r6l9m
0eb8c00bbb1a0       1fabf80a1273a       15 minutes ago      Running             kube-scheduler            0                   ef7a02ef641ee       kube-sched
uler-minikube
15df56cdfd814       364b3c3d9ec19       15 minutes ago      Running             kube-controller-manager   0                   20772974ad957       kube-contr
oller-manager-minikube
49933f149fe96       bec5f0e1e2eeb       15 minutes ago      Running             kube-apiserver            0                   3ddbf4de3665a       kube-apise
rver-minikube
887a634e38e3e       270fbeb697171       15 minutes ago      Running             etcd                      0                   c23285e13efd4       etcd-minik
ube
docker@minikube:~$
```

FIGURE 4.7 – `sudo crictl ps` dans le nœud : 8 conteneurs `Running`, tous dans le namespace `kube-system` : `kube-apiserver`, `etcd`, `kube-scheduler`, `kube-controller-manager`, `kube-proxy`, `coredns`, `storage-provisioner` et `kindnet-cni`.

Rôle de ces composants (voir Tableau 4.1) :

TABLEAU 4.1 – Rôle des composants du cluster observés dans le nœud

Composant	**Rôle**
`kube-apiserver`	Point d'entrée unique du cluster ; reçoit toutes les requêtes (dont celles de `kubectl`).
`etcd`	Base de données clé/valeur qui stocke l'état du cluster.
`kube-scheduler`	Choisit le nœud sur lequel chaque nouveau pod sera exécuté.
`kube-controller-manager`	Exécute les contrôleurs qui ramènent le cluster vers l'état désiré (ex. nombre de réplicas).
`kube-proxy`	Programme les règles réseau (iptables) qui implémentent les Services.
`coredns`	Serveur DNS interne du cluster (résolution des noms de services).
`kindnet-cni`	Plugin réseau (CNI) : attribue les adresses IP des pods et assure leur connectivité.
`storage-provisioner`	Addon Minikube qui crée automatiquement les volumes persistants demandés.

On quitte ensuite le nœud avec `exit` et on vérifie l'état du cluster depuis l'hôte :

```
exit
kubectl cluster-info
kubectl get nodes
```

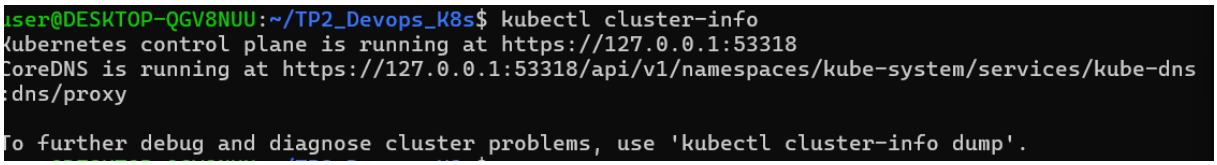


FIGURE 4.8 – `kubectl cluster-info` : le *control plane* (API server) répond sur `https://127.0.0.1:53318`, et **CoreDNS est en cours d'exécution** (`CoreDNS is running at .../kube-dns:dns/proxy`).

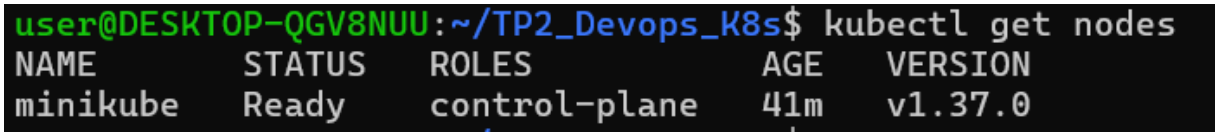


FIGURE 4.9 – `kubectl get nodes` : le nœud `minikube` est `Ready`, rôle `control-plane`, version v1.37.0.

CoreDNS est disponible et le nœud est prêt : le cluster est opérationnel et on peut commencer à créer des déploiements, des services, etc.

Remarque : pourquoi deux adresses IP différentes ?

TABLEAU 4.2 – Signification des deux adresses IP du cluster

Adresse	Signification
`127.0.0.1:53318`	Adresse utilisée par `kubectl` pour joindre le serveur d'API. Avec le driver Docker, le port 8443 de l'API server (dans le conteneur `minikube`) est **publié** sur un port aléatoire de la machine hôte (ici 53318). Minikube écrit cette adresse dans `~/.kube/config`.
`192.168.49.2`	Adresse IP du nœud Minikube (le conteneur `minikube`) sur le réseau Docker dédié.

`kubectl` passe donc par une redirection de port sur `localhost` au lieu de contacter directement l'IP du nœud :

```
kubectl --> 127.0.0.1:53318 (hôte WSL) --> redirection Docker --> 192.168.49.2:8443 (
    ↪ kube-apiserver dans le nœud)
```

Pods et namespaces

```
kubectl get pods
kubectl get namespaces
kubectl get pods --namespace=kube-system
```

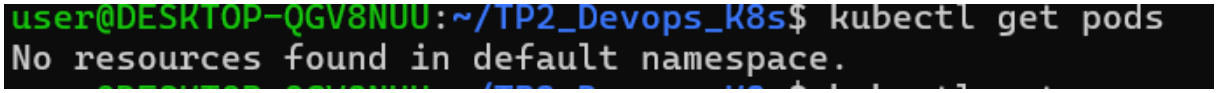


FIGURE 4.10 – `kubectl get pods` renvoie `No resources found in default namespace.`

Explication : par défaut, `kubectl get pods` ne liste que les pods du namespace courant, ici `default`. Un **pod** est la plus petite unité déployable de Kubernetes ; il contient un ou plusieurs conteneurs qui partagent le même réseau (même adresse IP) et éventuellement des volumes. Le namespace `default` existe mais on n'y a encore déployé aucune application : Kubernetes indique donc qu'il n'y a aucune ressource.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   43m
kube-node-lease   Active   43m
kube-public       Active   43m
kube-system       Active   43m
```

FIGURE 4.11 – `kubectl get namespaces` : les 4 namespaces créés par défaut, tous `Active` : `default` (nos applications), `kube-system` (composants du cluster), `kube-public` (ressources lisibles par tous) et `kube-node-lease` (signaux de vie des nœuds).

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl get pods --namespace=kube-sys
tem
NAME                               READY   STATUS    RESTARTS   AGE
coredns-559f6c778d-hltn2           1/1     Running   0          46m
etcd-minikube                      1/1     Running   0          46m
kindnet-5qhwr                      1/1     Running   0          46m
kube-apiserver-minikube            1/1     Running   0          46m
kube-controller-manager-minikube   1/1     Running   0          46m
kube-proxy-r6l9m                   1/1     Running   0          46m
kube-scheduler-minikube            1/1     Running   0          46m
storage-provisioner                1/1     Running   0          46m
```

FIGURE 4.12 – `kubectl get pods --namespace=kube-system` : les 8 pods système (`coredns`, `etcd-minikube`, `kindnet`, `kube-apiserver-minikube`, `kube-controller-manager-minikube`, `kube-proxy`, `kube-scheduler-minikube`, `storage-provisioner`) sont `1/1 Running`, sans redémarrage. Ce sont les mêmes que les conteneurs vus avec `crictl ps`.

5

Création d'un pod

5.1 Création d'un pod unique

On a créé un pod nommé `nginx` dans le namespace `default` :

```
kubectl run nginx --image=nginx
kubectl get pods
```

Ce pod utilise l'image officielle `nginx`, qui contient le serveur web Nginx. Après le téléchargement de l'image et le démarrage du conteneur, le pod passe de l'état `ContainerCreating` à l'état `Running`.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl run nginx --image=nginx
pod/nginx created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl get pods
NAME    READY   STATUS              RESTARTS   AGE
nginx   0/1     ContainerCreating   0          13s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl get pods
NAME    READY   STATUS              RESTARTS   AGE
nginx   0/1     ContainerCreating   0          26s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl get pods
NAME    READY   STATUS              RESTARTS   AGE
nginx   0/1     ContainerCreating   0          30s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
nginx   1/1     Running   0          41s
```

FIGURE 5.1 – `kubectl run nginx --image=nginx` (`pod/nginx created`) puis plusieurs `kubectl get pods` : le pod reste `0/1 ContainerCreating` pendant le téléchargement de l'image (13 s, 26 s, 30 s), puis passe à `1/1 Running` (41 s).

La commande suivante permet d'obtenir des informations détaillées sur le pod :

```
kubectl describe pod nginx
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl describe pod nginx
Name:             nginx
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Thu, 15 Oct 2026 19:02:30 +0100
Labels:           run=nginx
Annotations:      <none>
Status:           Running
IP:               10.244.0.3
IPs:
  IP:  10.244.0.3
Containers:
  nginx:
    Container ID:   containerd://60463b7900a612f0c1bdd291f85fc216bad964b9f17
f785c216a0c7955cf083a
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:332d0beea97cff8c0d5d27976
c0778a4b00345b9d74c2feaaf8efde548314948
    Port:           <none>
    Host Port:      <none>
    State:          Running
```

FIGURE 5.2 – `kubectl describe pod nginx` (début de la sortie) : nœud `minikube/192.168.49.2`, démarrage le 15/10/2026 à 19 :02 :30, label `run=nginx`, statut `Running`, IP `10.244.0.3`, `Container ID: containerd://60463b79...` et `Image ID: docker.io/library/nginx@sha256:332d0bee...`.

Les principales informations obtenues sont les suivantes :

— **Identification du pod :**

- `Name: nginx` : nom du pod créé.
- `Namespace: default` : namespace dans lequel le pod est déployé.
- `Node: minikube/192.168.49.2` : nœud sur lequel le pod s'exécute (le seul nœud du cluster).
- `Start Time` : date et heure de démarrage du pod.
- `Labels: run=nginx` : étiquette ajoutée automatiquement par `kubectl run`.

— **État et réseau :**

- `Status: Running` : le pod est en cours d'exécution.
- `IP: 10.244.0.3` : adresse IP interne attribuée au pod, prise dans la plage réseau des pods (`10.244.0.0/16`).
- `Ready: True` : le conteneur est prêt à recevoir du trafic.
- `Restart Count: 0` : le conteneur n'a pas redémarré depuis son lancement.

— **Informations sur le conteneur :**

- `Container ID: containerd://...` : identifiant unique du conteneur ; le préfixe `containerd://` confirme que le runtime utilisé est containerd.
- `Image: nginx` : image utilisée pour créer le conteneur.
- `Image ID` : référence de l'image téléchargée, avec son empreinte (digest SHA-256).
- `State: Running` : état actuel du conteneur.

— **Conditions du pod :**

- `PodScheduled: True` : le pod a été affecté à un nœud.
- `PodReadyToStartContainers: True` : l'environnement du pod (sandbox, réseau) est prêt.
- `Initialized: True` : l'initialisation du pod est terminée.
- `ContainersReady: True` : tous les conteneurs du pod sont prêts.
- `Ready: True` : le pod peut recevoir du trafic.

— **Volumes :**

- `kube-api-access-f5s88` : volume projeté monté automatiquement, qui contient le jeton (token) du ServiceAccount, le certificat de l'autorité de certification du cluster (`kube-root-ca.crt`, issu d'un ConfigMap) et le namespace. Il permet au

pod de s'authentifier auprès de l'API Kubernetes.

— **Événements (Events) :** ils retracent le cycle de vie du pod :

  — `Scheduled` : le scheduler a affecté le pod au nœud `minikube` ;

  — `Pulling` : le kubelet commence à télécharger l'image `nginx` ;

  — `Pulled` : téléchargement terminé ;

  — `Created` : le conteneur a été créé ;

  — `Started` : le conteneur a démarré.

5.2 Exploration du pod

Dans le nœud, un nouveau conteneur `nginx` est apparu :

```
minikube ssh
sudo crictl ps | grep nginx
```

```
docker@minikube:~$ sudo crictl ps | grep nginx
60463b7900a61       ca5e66f340c76       53 minutes ago      Running
    nginx                       0                   838c714bc8e4f       nginx
                               default
```

FIGURE 5.3 – `sudo crictl ps | grep nginx` dans le nœud : le conteneur `nginx` (ID `60463b7900a61`, le même que le `Container ID` de `describe pod`) est `Running` dans le pod `nginx` du namespace `default`.

On peut entrer dans le conteneur avec son identifiant (ou le début de celui-ci). Là où le formateur utilise `docker exec`, on utilise `crictl exec` :

```
sudo crictl exec -it 60463b7900a61 sh
hostname
hostname -i
curl 10.244.0.3
```



FIGURE 5.4 – Shell à l'intérieur du conteneur nginx : `hostname` renvoie `nginx` (le nom du pod), `hostname -i` renvoie `10.244.0.3` (l'IP du pod), et `curl 10.244.0.3` renvoie le code HTML de la page « Welcome to nginx ! ».

Le conteneur `pause`. **Chaque pod possède un conteneur supplémentaire, le conteneur pause **(ou *sandbox*). C'est lui qui crée et garde ouverts les namespaces Linux du pod, en particulier le namespace réseau. Tous les conteneurs du pod rejoignent ce namespace réseau :** ils partagent donc la même adresse IP** (`10.244.0.3`) et peuvent communiquer entre eux via `localhost`.

> **Remarque (différence avec la vidéo).** Dans la vidéo, le nœud utilise **Docker** comme runtime : le conteneur pause apparaît donc directement dans `docker ps`. C'est aussi ce que voient les étudiants qui utilisent un driver VirtualBox ou Hyper-V avec le runtime Docker. Notre nœud utilise **containerd** : le conteneur pause existe toujours, mais il n'apparaît **ni dans `crictl ps`, ni dans `crictl ps -a`, qui ne listent que les conteneurs applicatifs. Il est géré comme le** sandbox** du pod et se voit avec une autre commande : `sudo crictl pods`.

```
sudo crictl ps -a --name nginx     # le conteneur nginx et l'ID de son pod (POD ID)
sudo crictl pods --name nginx      # le sandbox (conteneur pause) du pod
sudo crictl pods -q | wc -l        # nombre de sandboxes sur le nœud
ps aux | grep -c [/]pause          # nombre de processus /pause
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ minikube ssh
docker@minikube:~$ sudo crictl ps -a --name nginx | awk 'NR==1{printf "%-15s
 %-8s %s\n","CONTAINER","NAME","POD ID"} NR>1{printf "%-15s %-8s %s\n",$1,$(
NF-4),$(NF-2)}'
CONTAINER       NAME     POD ID
3c5910b61ed61   nginx    4f40cf76de942
docker@minikube:~$ sudo crictl pods --name nginx | awk 'NR==1{printf "%-15s
%-8s %s\n","POD ID","STATE","NAME"} NR>1{printf "%-15s %-8s %s\n",$1,$(NF-4)
,$(NF-3)}'
POD ID          STATE    NAME
4f40cf76de942   Ready    nginx
docker@minikube:~$ sudo crictl pods -q | wc -l
11
docker@minikube:~$ ps aux | grep -c [/]pause
11
```

FIGURE 5.5 – Pour un pod `nginx` recréé pour l'occasion : le conteneur `nginx` (`3c5910b61ed61`) appartient au pod `4f40cf76de942`, et c'est exactement ce `POD ID` que `crictl pods` liste comme sandbox, à l'état `Ready`. Le nœud compte 11 sandboxes et 11 processus `/pause` : un conteneur pause par pod.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl get pods -o wide
NAME    READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED
 NODE   READINESS GATES
nginx   1/1     Running   0          60m   10.244.0.3   minikube   <none>
        <none>
```

FIGURE 5.6 – `kubectl get pods -o wide` : le pod `nginx` a l'IP `10.244.0.3` et s'exécute sur le nœud `minikube`. C'est la même adresse que celle renvoyée par `hostname -i` dans le conteneur : tous les conteneurs d'un pod partagent cette IP.

Accès depuis l'extérieur du cluster. Depuis l'hôte (WSL), la commande `curl 10.244.0.3` échoue :

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ curl 10.244.0.3
^C
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$
```

FIGURE 5.7 – `curl 10.244.0.3` lancé depuis WSL (hors du nœud) : aucune réponse, la commande est interrompue avec Ctrl+C.

L'adresse `10.244.0.3` est l'IP du pod sur le **réseau interne du cluster**. Elle est joignable depuis l'intérieur du cluster (autres pods, nœud Minikube), mais pas depuis l'hôte, qui n'a aucune route vers ce réseau. Pour accéder à un pod depuis l'extérieur, il faut utiliser `kubectl port-forward`, un Service de type NodePort ou LoadBalancer, ou un Ingress (voir chapitres 7 et 9).

Suppression du pod de test :

```
kubectl delete pod nginx
kubectl get pods
```

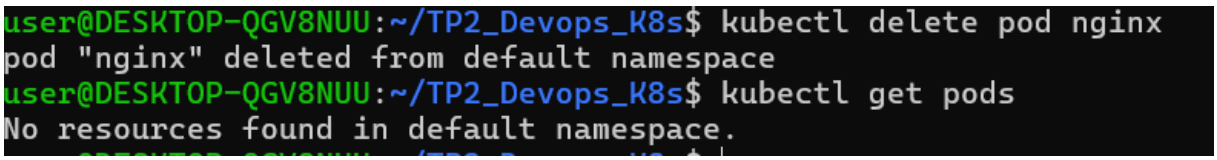


FIGURE 5.8 – `kubectl delete pod nginx` (`pod "nginx" deleted from default namespace`) puis `kubectl get pods` : il n'y a plus aucun pod dans le namespace `default`.

5.3 Création d'un alias pour kubectl

Pour gagner du temps, on crée l'alias `k` :

```
alias k="kubectl"
k get pods
```

(Pour rendre l'alias permanent, on l'ajoute dans `~/.bashrc`.)

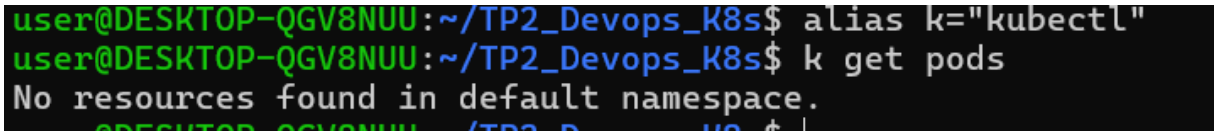


FIGURE 5.9 – Création de l'alias `k` puis `k get pods` : l'alias fonctionne (`No resources found in default namespace.`).

6

Création d'un déploiement

6.1 Création et exploration d'un déploiement

Créer les pods un par un n'est pas pratique pour faire évoluer une application (*scaling*). La méthode recommandée est de créer un **Deployment** : on décrit l'état désiré (image, nombre de réplicas) et Kubernetes crée et maintient automatiquement les pods correspondants. On crée ensuite un **Service** pour accéder à ces pods.

```
k create deployment nginx-deployment --image=nginx
k get deployment
k get pods
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k create deployment nginx-deployment
--image=nginx
deployment.apps/nginx-deployment created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get deployment
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   1/1     1            1           13s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-5467ddfc68-stlxj   1/1     Running   0          19s
```

FIGURE 6.1 – Création du déploiement `nginx-deployment` : `k get deployment` affiche `1/1` prêt, et `k get pods` montre le pod `nginx-deployment-5467ddfc68-stlxj`, `Running`, créé **automatiquement** par le déploiement.

Remarque : ce pod n'a pas été créé manuellement : il a été créé par le déploiement (plus précisément par son ReplicaSet) qui le gère. Si on le supprime, il est recréé automatiquement.

```
k describe deployment nginx-deployment
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k describe deployment nginx-deploymen
t
Name:                   nginx-deployment
Namespace:              default
CreationTimestamp:      Thu, 15 Oct 2026 20:18:40 +0100
Labels:                 app=nginx-deployment
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=nginx-deployment
Replicas:               1 desired | 1 updated | 1 total | 1 available | 0 un
available
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx-deployment
  Containers:
```

FIGURE 6.2 – `k describe deployment nginx-deployment` : label `app=nginx-deployment`, `Selector: app=nginx-deployment`, `Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable`, `StrategyType: RollingUpdate` (`25% max unavailable, 25% max surge`).

```
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   nginx-deployment-5467ddfc68 (1/1 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  86s   deployment-controller  Scaled up replica
```

FIGURE 6.3 – Suite de la sortie : conditions `Available` et `Progressing` à `True`, `NewReplicaSet: nginx-deployment-5467ddfc68 (1/1 replicas created)`, et l'événement `ScalingReplicaSet` du `deployment-controller`.

Champs importants :

— **Selector : `app=nginx-deployment`** : le *label* qui permet au déploiement (et à son ReplicaSet) d'identifier les pods qu'il gère. Tous les pods créés portent ce label.

— **Replicas : `1 desired | 1 updated | 1 total | 1 available | 0 unavailable`** : nombre de pods souhaités et état réel. Le contrôleur fait en sorte que le nombre réel soit toujours égal au nombre souhaité.

— **StrategyType : `RollingUpdate`** : stratégie de mise à jour par défaut (les pods sont remplacés progressivement, sans interruption de service).

— **NewReplicaSet : `nginx-deployment-5467ddfc68` : le ReplicaSet est l'objet intermédiaire créé par le déploiement. C'est lui qui maintient le nombre de pods. La hiérarchie est : Deployment → ReplicaSet → Pods**.

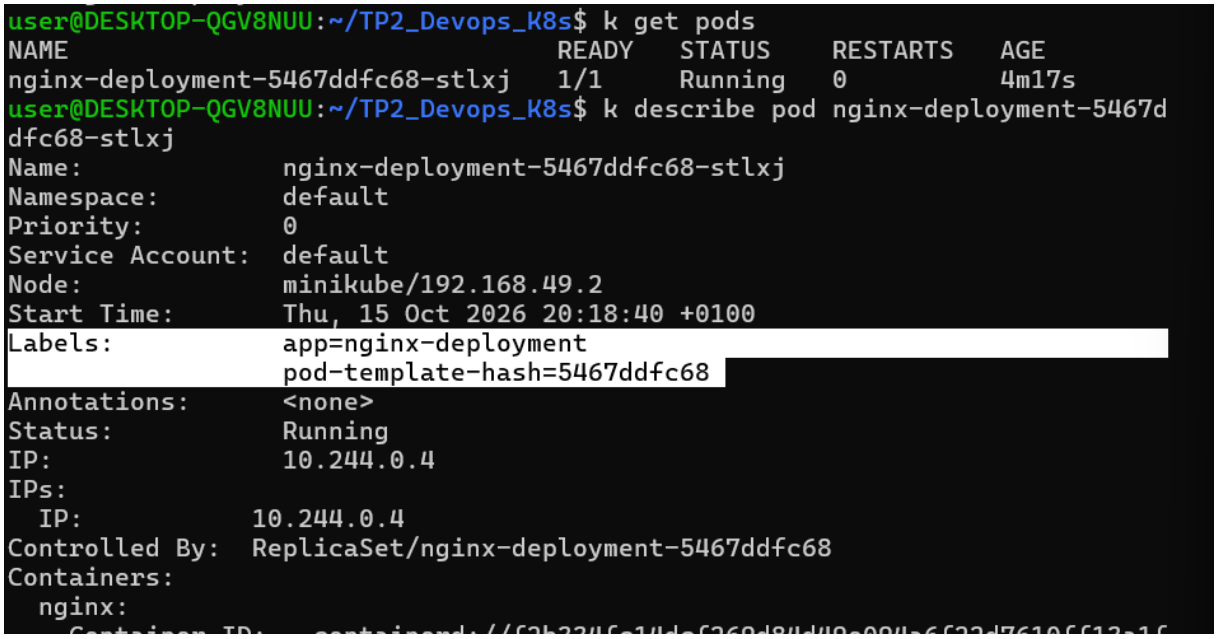


FIGURE 6.4 – `k describe pod nginx-deployment-5467ddfc68-stlxj` : labels `app=nginx-deployment` et `pod-template-hash=5467ddfc68`, IP `10.244.0.4`, et `Controlled By: ReplicaSet/nginx-deployment-5467ddfc68` : le pod est géré par le ReplicaSet du déploiement. Le `pod-template-hash` est le hash qui apparaît dans le nom du ReplicaSet et des pods.

6.2 Mise à l'échelle (scaling) du déploiement

```
k scale deployment nginx-deployment --replicas=5
k get pods
k get pods -o wide
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k scale deployment nginx-deployment --
-replicas=5
deployment.apps/nginx-deployment scaled
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get pods
NAME                                READY   STATUS              RESTARTS   A
GE
nginx-deployment-5467ddfc68-92mpp   0/1     ContainerCreating   0          5
s
nginx-deployment-5467ddfc68-bx666   1/1     Running             0          5
s
nginx-deployment-5467ddfc68-n2qjh   1/1     Running             0          5
s
nginx-deployment-5467ddfc68-stlxj   1/1     Running             0          7
m25s
nginx-deployment-5467ddfc68-zgz7c   0/1     ContainerCreating   0          5
s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-5467ddfc68-92mpp   1/1     Running   0          9s
nginx-deployment-5467ddfc68-bx666   1/1     Running   0          9s
nginx-deployment-5467ddfc68-n2qjh   1/1     Running   0          9s
nginx-deployment-5467ddfc68-stlxj   1/1     Running   0          7m29s
nginx-deployment-5467ddfc68-zgz7c   1/1     Running   0          9s
```

FIGURE 6.5 – Après `k scale ... --replicas=5` : 4 nouveaux pods (`92mpp`, `bx666`, `n2qjh`, `zgz7c`) sont créés à côté de `stlxj`, d'abord en `ContainerCreating` puis tous `Running`. Ils ont tous le même préfixe `nginx-deployment-5467ddfc68` (nom du ReplicaSet) ; seul le suffixe aléatoire final change.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE     IP
          NODE       NOMINATED NODE   READINESS GATES
nginx-deployment-5467ddfc68-92mpp   1/1     Running   0          111s    10.
244.0.7   minikube   <none>           <none>
nginx-deployment-5467ddfc68-bx666   1/1     Running   0          111s    10.
244.0.5   minikube   <none>           <none>
nginx-deployment-5467ddfc68-n2qjh   1/1     Running   0          111s    10.
244.0.6   minikube   <none>           <none>
nginx-deployment-5467ddfc68-stlxj   1/1     Running   0          9m11s   10.
244.0.4   minikube   <none>           <none>
nginx-deployment-5467ddfc68-zgz7c   1/1     Running   0          111s    10.
244.0.8   minikube   <none>           <none>
```

FIGURE 6.6 – `k get pods -o wide` : chaque pod a une adresse IP différente (`10.244.0.4` à `10.244.0.8`), attribuée automatiquement par le réseau du cluster (plugin CNI) ; on ne l'a pas choisie.

Réduction du nombre de réplicas (*downscale*) :

```
k scale deployment nginx-deployment --replicas=3
k get pods -o wide
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k scale deployment nginx-deployment --replicas=3
deployment.apps/nginx-deployment scaled
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE     IP
          NODE       NOMINATED NODE   READINESS GATES
nginx-deployment-5467ddfc68-92mpp   1/1     Running   0          2m45s   10.
244.0.7   minikube   <none>           <none>
nginx-deployment-5467ddfc68-bx666   1/1     Running   0          2m45s   10.
244.0.5   minikube   <none>           <none>
nginx-deployment-5467ddfc68-stlxj   1/1     Running   0          10m     10.
244.0.4   minikube   <none>           <none>
```

FIGURE 6.7 – Après le passage à 3 réplicas : 2 pods ont été supprimés ; il reste `92mpp` (`10.244.0.7`), `bx666` (`10.244.0.5`) et `stlxj` (`10.244.0.4`).

6.3 Connexion à un pod via son adresse IP

Depuis WSL, les IP des pods ne sont pas joignables (comme pour le pod du chapitre 5). Depuis le nœud :

```
minikube ssh
curl 10.244.0.5
```

```
docker@minikube:~$ curl 10.244.0.5
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
```

FIGURE 6.8 – Depuis le nœud (`docker@minikube`), `curl 10.244.0.5` renvoie la page « Welcome to nginx ! » servie par l'un des pods du déploiement.

Limite de cette approche : pour joindre les pods, on a utilisé leurs adresses IP. Or ces adresses sont attribuées automatiquement par Kubernetes et **changent** dès qu'un pod est recréé (scaling, crash, mise à jour). De plus, un client devrait choisir lui-même un pod parmi plusieurs. La solution est de créer un **Service**, qui fournit une adresse IP et un nom DNS **stables** devant l'ensemble des pods du déploiement, et qui répartit le trafic entre eux.

7

Création d'un service

7.1 Qu'est-ce qu'un Service ?

Un Service est un objet Kubernetes qui expose un ensemble de pods (sélectionnés par leurs labels) derrière une adresse unique et stable, et qui répartit la charge entre eux. Principaux types (voir Tableau 7.1) :

Tableau 7.1 – Principaux types de services Kubernetes

Type	**Description**
ClusterIP (par défaut)	Adresse IP virtuelle unique, joignable **uniquement depuis l'intérieur du cluster**.
NodePort	En plus du ClusterIP, ouvre un port (plage 30000–32767) sur **chaque nœud** : le service devient joignable via `<IP_du_nœud>:<NodePort>`.
LoadBalancer	En plus du NodePort, demande au fournisseur cloud (AWS, GCP, Azure…) un répartiteur de charge externe avec une **adresse IP externe** unique.

7.2 Création et exploration d'un service ClusterIP

```
k expose deployment nginx-deployment --port=8080 --target-port=80
k get services
k describe svc nginx-deployment
```

— `--port=8080` : port **du service** (sur l'adresse ClusterIP) ;

— `--target-port=80` : port **des conteneurs** vers lequel le trafic est redirigé (nginx écoute sur le port 80).

> Remarque : le port 8080 n'est pas un port « externe ». Un service ClusterIP reste interne au cluster ; 8080 est simplement le port sur lequel le service écoute.

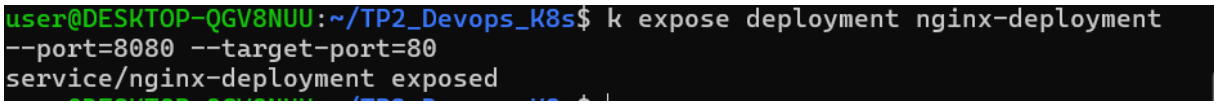


FIGURE 7.1 – `k expose deployment nginx-deployment --port=8080 --target-port=80` : `service/nginx-deployment exposed`.

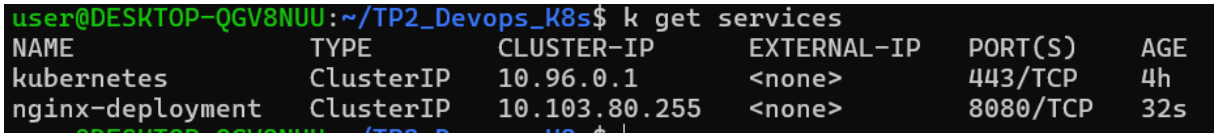


FIGURE 7.2 – `k get services` : le service `nginx-deployment` est de type `ClusterIP`, avec l'adresse `10.103.80.255` et le port `8080/TCP`. Le service `kubernetes` (`10.96.0.1`, `443/TCP`) correspond à l'API server.

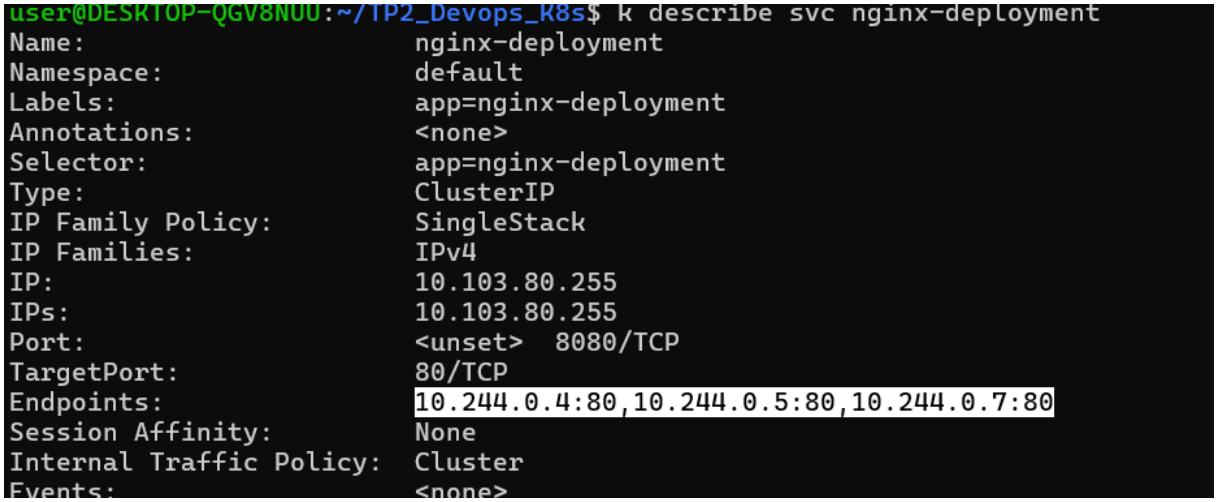


FIGURE 7.3 – `k describe svc nginx-deployment` : `Selector: app=nginx-deployment`, `Type: ClusterIP`, `IP: 10.103.80.255`, `Port: 8080/TCP`, `TargetPort: 80/TCP` et `Endpoints: 10.244.0.4:80, 10.244.0.5:80, 10.244.0.7:80` : ce sont exactement les IP des 3 pods du déploiement (voir 6.2).

7.3 Connexion au déploiement via le service ClusterIP

Tentative depuis l'hôte (WSL) :

```
curl 10.103.80.255:8080
```

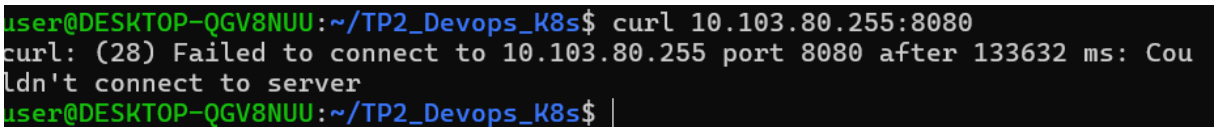


FIGURE 7.4 – `curl 10.103.80.255:8080` depuis WSL : échec après environ 2 minutes (`curl: (28) Failed to connect to 10.103.80.255 port 8080 after 133632 ms: Couldn't connect to server`).

La requête est envoyée depuis l'hôte, qui se trouve en dehors du réseau du cluster. `10.103.80.255` est une adresse ClusterIP, c'est-à-dire une **adresse virtuelle** : elle n'est attribuée à aucune interface réseau et n'existe que sous forme de règles (iptables) programmées par `kube-proxy` sur les nœuds du cluster.

Connexion depuis le nœud :

```
minikube ssh
curl 10.103.80.255:8080
```

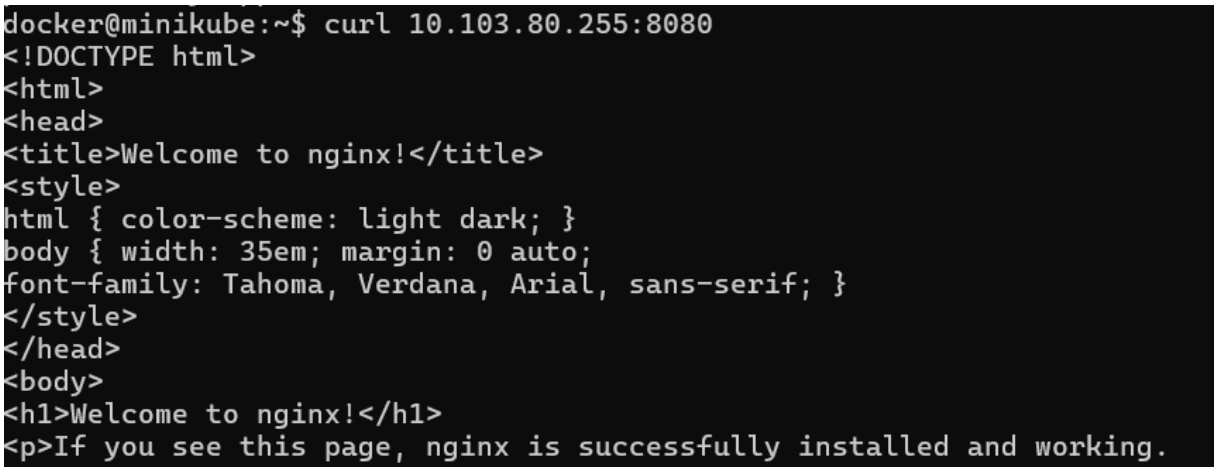


FIGURE 7.5 – Depuis le nœud (`docker@minikube`), `curl 10.103.80.255:8080` renvoie la page « Welcome to nginx ! ».

La réponse est fournie par **l'un des pods** du déploiement (l'un des 3 endpoints vus avec `describe svc`). Le rôle du ClusterIP est de **répartir la charge** entre les pods du déploiement : chaque nouvelle connexion est redirigée vers l'un des endpoints.

Remarque : avec l'image nginx, tous les pods renvoient exactement la même page ; on ne peut donc pas voir quel pod a répondu. C'est pour cela qu'au chapitre 9 on crée une application qui renvoie le nom d'hôte (*hostname*) du pod.

8

Suppression du déploiement et du service

8.1 Suppression

```
k delete deployment nginx-deployment
k delete service nginx-deployment
k get all
```

Supprimer le déploiement supprime aussi son ReplicaSet et tous ses pods. Le service, lui, est un objet indépendant : il doit être supprimé séparément. Après ces deux commandes, `k get all` n'affiche plus que le service système `kubernetes`.

9

Exécution d'une application Node.js avec Kubernetes

Objectif : créer une image Docker personnalisée, la publier sur Docker Hub, puis créer un déploiement et des services à partir de cette image. L'application est un serveur web qui répond avec le **nom d'hôte** du serveur. Comme le hostname d'un conteneur est le nom de son pod, cela permet de voir **quel pod** a traité chaque requête.

9.1 Création de l'application web Node.js

Initialisation du projet et installation d'Express :

```
mkdir k8s-web-hello && cd k8s-web-hello
npm init -y
npm install express
```

Fichier `index.mjs` :

```
import express from 'express'
import os from 'os'

const app = express()
```

```
const PORT = 3000

app.get("/", (req, res) => {
  const helloMessage = `<h1>VERSION 2: Hello from the ${os.hostname()}</h1>`
  console.log(helloMessage)
  res.send(helloMessage)
})

app.listen(PORT, () => {
  console.log(`Web server is listening at port ${PORT}`)
})
```

Dans `package.json`, on ajoute le script de démarrage : `"start": "node index.mjs"`.

> Le message contient déjà « VERSION 2 » : c'est la version de notre application au moment des sections 9.x (la vidéo n'ajoute ce préfixe qu'en 10.1).

9.2 Dockerisation de l'application

Fichier `Dockerfile` :

```
FROM --platform=linux/amd64 node:alpine
WORKDIR /app
EXPOSE 3000
COPY package.json package-lock.json ./
RUN npm install
COPY . ./
CMD ["npm", "start"]
```

Construction de l'image :

```
docker build . -t hanine169/k8s-web-hello
docker images | grep k8s-web
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ docker build . -t hanin
e169/k8s-web-hello
[+] Building 41.2s (10/10) FINISHED                             docker:default
 => [internal] load build definition from Dockerfile                       0.1s
```

FIGURE 9.1 – `docker build . -t hanine169/k8s-web-hello` : `Building 41.2s (10/10) FINISHED`, toutes les étapes du Dockerfile ont été exécutées avec succès.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ docker images | grep k8
s-web
hanine169/k8s-web-hello:latest
                          7dc00e85a8dd        268MB         65.5MB
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$
```

FIGURE 9.2 – `docker images | grep k8s-web` : l'image `hanine169/k8s-web-hello:latest` (ID `7dc00e85a8dd`, 268 Mo, 65,5 Mo compressée) est présente localement.

9.3 Publication de l'image sur Docker Hub

```
docker login
docker push hanine169/k8s-web-hello
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ docker push hanine169/k
8s-web-hello
Using default tag: latest
The push refers to repository [docker.io/hanine169/k8s-web-hello]
2af57b4eb954: Pushed
d428a8ca80cd: Pushed
5a84e4ea9a0a: Pushed
ce75b3b95c4c: Pushed
e2de96513ba9: Pushed
d8b8ea226d22: Pushed
58578844f8d0: Pushed
dd6eb71d102b: Pushed
latest: digest: sha256:7dc00e85a8ddb14594cbc11cd427490639c3595a3ec9c809c10fb
83147e01a59 size: 856
```

FIGURE 9.3 – `docker push hanine169/k8s-web-hello` : tag par défaut `latest`, les 8 couches de l'image sont envoyées (`Pushed`) vers `docker.io/hanine169/k8s-web-hello`, avec le digest `sha256:7dc00e85...`.

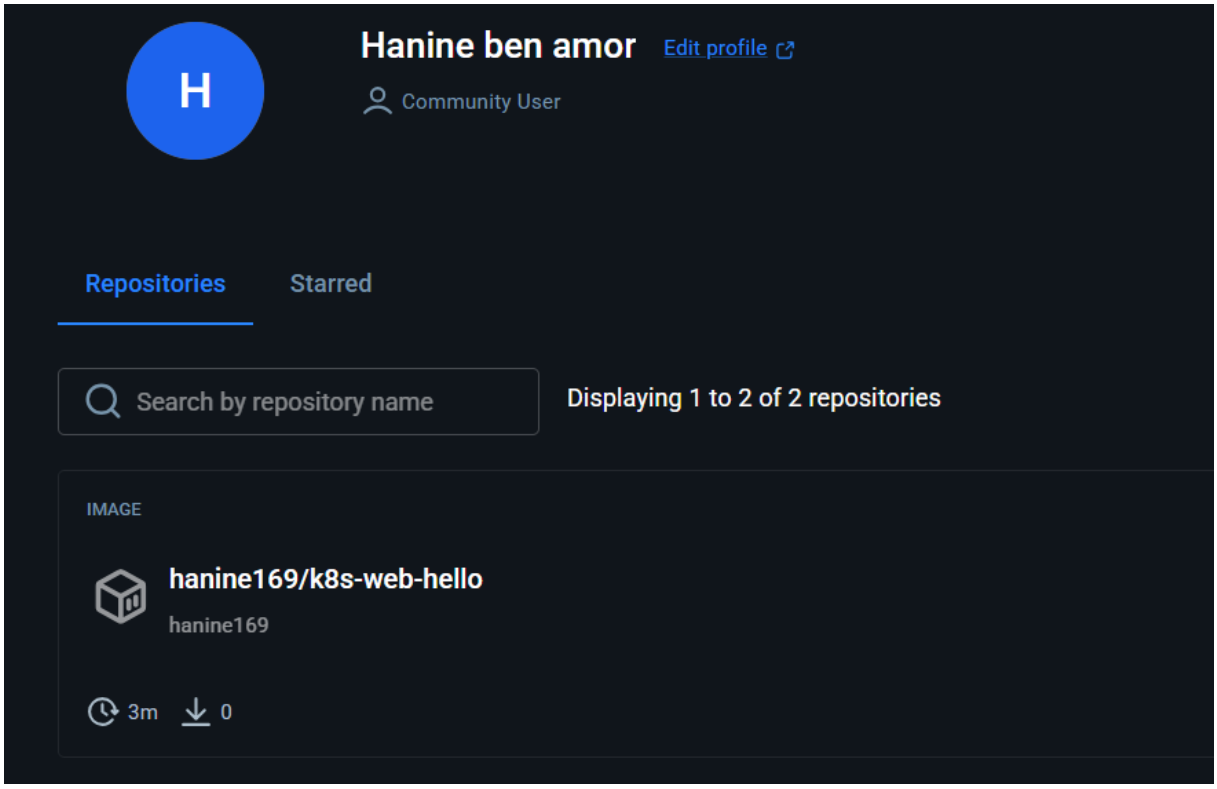


FIGURE 9.4 – Le dépôt `hanine169/k8s-web-hello` est visible dans notre compte Docker Hub (poussé 3 minutes plus tôt).

L'image est maintenant publique : le nœud Minikube peut la télécharger comme n'importe quelle image officielle.

9.4 Création d'un déploiement à partir de l'image personnalisée

```
k create deployment k8s-web-hello --image=hanine169/k8s-web-hello
k get pods
k expose deployment k8s-web-hello --port=3000
k get svc
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k create deployment
error: required flag(s) "image" not set
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k create deployment k8s
-web-hello --image=hanine169/k8s-web-hello
deployment.apps/k8s-web-hello created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k get pods
NAME                             READY   STATUS    RESTARTS   AGE
k8s-web-hello-589649bdcd-qj7bm   1/1     Running   0          36s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k expose deployment k8s
-web-hello --port=3000
service/k8s-web-hello exposed
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k get svc
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)    AGE
k8s-web-hello   ClusterIP   10.96.27.89   <none>        3000/TCP   6s
kubernetes      ClusterIP   10.96.0.1     <none>        443/TCP    5h3m
```

FIGURE 9.5 – Création du déploiement `k8s-web-hello` à partir de notre image (le premier essai sans `--image` est refusé : `error: required flag(s) "image" not set`), pod `k8s-web-hello-589649bdcd-qj7bm Running`, puis création du service ClusterIP `k8s-web-hello` : adresse `10.96.27.89`, port `3000/TCP`.

Test depuis le nœud :

```
minikube ssh
curl 10.96.27.89:3000
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ minikube ssh
Linux minikube 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu
Jun 18 21:54:43 UTC 2026 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
docker@minikube:~$ curl 10.96.27.89:3000
<h1>VERSION 2: Hello from the k8s-web-hello-589649bdcd-qj7bm</h1>docker@mini
kube:~$
```

FIGURE 9.6 – Depuis le nœud, `curl 10.96.27.89:3000` renvoie « VERSION 2 : Hello from the k8s-web-hello-589649bdcd-qj7bm », c'est-à-dire le nom du pod qui a répondu.

9.5 Mise à l'échelle du déploiement de l'image personnalisée

```
k scale deployment k8s-web-hello --replicas=4
k get pods
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k scale deployment k8s-
web-hello --replicas=4
deployment.apps/k8s-web-hello scaled
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k get pods
NAME                              READY   STATUS    RESTARTS   AGE
k8s-web-hello-589649bdcd-5z85f    1/1     Running   0          5s
k8s-web-hello-589649bdcd-kmqh4    1/1     Running   0          5s
k8s-web-hello-589649bdcd-mdrwn    1/1     Running   0          5s
k8s-web-hello-589649bdcd-qj7bm    1/1     Running   0          16m
```

FIGURE 9.7 – Après `k scale ... --replicas=4` : 3 nouveaux pods (`5z85f`, `kmqh4`, `mdrwn`, âge 5 s) à côté de `qj7bm` (16 min), tous `1/1 Running`.

> Remarque : on passe à 4 **pods** (réplicas), pas à 4 nœuds. Le cluster Minikube ne contient toujours qu'un seul nœud.

Depuis le nœud, on envoie plusieurs requêtes **toujours vers la même adresse ClusterIP du service** :

```
minikube ssh
curl 10.99.106.158:3000; echo
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ minikube ssh
docker@minikube:~$ curl 10.99.106.158:3000; echo
<h1>VERSION 2: Hello from the k8s-web-hello-589649bdcd-kmqh4</h1>
docker@minikube:~$ curl 10.99.106.158:3000; echo
<h1>VERSION 2: Hello from the k8s-web-hello-589649bdcd-5z85f</h1>
docker@minikube:~$ curl 10.99.106.158:3000; echo
<h1>VERSION 2: Hello from the k8s-web-hello-589649bdcd-kmqh4</h1>
docker@minikube:~$ curl 10.99.106.158:3000; echo
<h1>VERSION 2: Hello from the k8s-web-hello-589649bdcd-5z85f</h1>
docker@minikube:~$ curl 10.99.106.158:3000; echo
<h1>VERSION 2: Hello from the k8s-web-hello-589649bdcd-qj7bm</h1>
docker@minikube:~$ curl 10.99.106.158:3000; echo
<h1>VERSION 2: Hello from the k8s-web-hello-589649bdcd-mdrwn</h1>
docker@minikube:~$ curl 10.99.106.158:3000; echo
<h1>VERSION 2: Hello from the k8s-web-hello-589649bdcd-mdrwn</h1>
docker@minikube:~$ curl 10.99.106.158:3000; echo
<h1>VERSION 2: Hello from the k8s-web-hello-589649bdcd-kmqh4</h1>
docker@minikube:~$
```

FIGURE 9.8 – Depuis le nœud, 8 requêtes successives vers la même adresse ClusterIP du service : les réponses proviennent des 4 pods différents (`kmqh4`, `5z85f`, `qj7bm`, `mdrwn`). Le service répartit la charge entre les **pods**.

> Capture refaite après coup sur le même déploiement (4 pods) : l'adresse `10.99.106.158` est le ClusterIP du service de la section 9.7 (un service LoadBalancer possède aussi un ClusterIP).

Suppression du service ClusterIP :

```
k delete svc k8s-web-hello
curl 10.96.27.89:3000; echo     # depuis le nœud
```

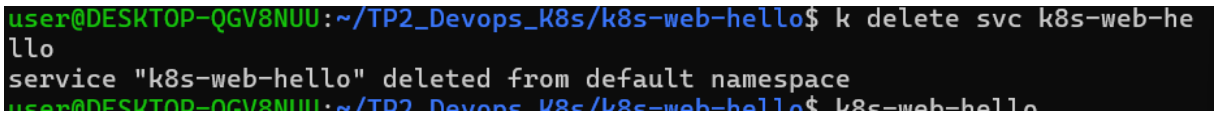


FIGURE 9.9 – `k delete svc k8s-web-hello` : `service "k8s-web-hello" deleted from default namespace`.



FIGURE 9.10 – Depuis le nœud, `curl 10.96.27.89:3000` ne reçoit plus de réponse (interrompu avec Ctrl+C) : l'adresse ClusterIP n'existait que tant que le service existait. Les pods, eux, tournent toujours.

9.6 Création d'un service NodePort

```
k expose deployment k8s-web-hello --type=NodePort --port=3000
k get svc
```

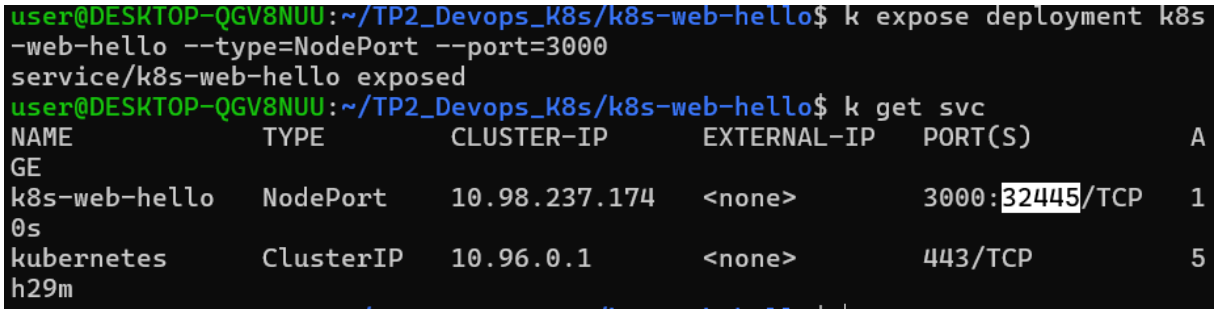


FIGURE 9.11 – `k get svc` : le service `k8s-web-hello` est de type `NodePort` (ClusterIP `10.98.237.174`) avec `3000:32445/TCP`. Le port **32445** a été choisi aléatoirement par Kubernetes dans la plage 30000–32767.

Un service NodePort ouvre ce port sur le nœud : l'application est accessible à l'adresse `<IP_du_nœud>:<NodePort>`, soit `192.168.49.2:32445`.

Comme on travaille sous WSL avec le driver Docker, le réseau du nœud Minikube n'est pas joignable directement depuis le navigateur Windows. On a donc utilisé Minikube pour obtenir une URL d'accès au service :

```
minikube service k8s-web-hello --url
```

> Avec le driver Docker sous WSL, l'URL fournie par Minikube n'est pas une URL publique : c'est un tunnel local sur `http://127.0.0.1:<port>`, actif tant que la commande `minikube service` tourne (« the terminal needs to be open to run it »).

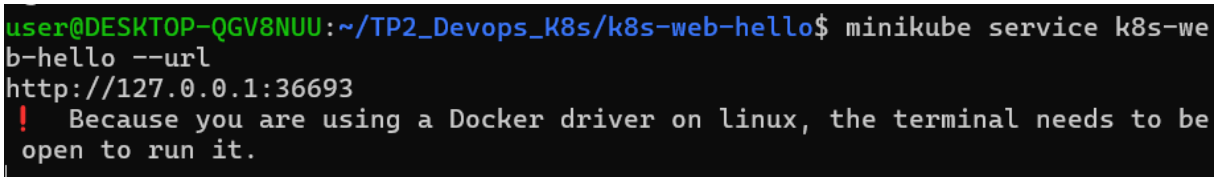


FIGURE 9.12 – `minikube service k8s-web-hello --url` renvoie `http://127.0.0.1:36693`, avec l'avertissement « Because you are using a Docker driver on linux, the terminal needs to be open to run it ».



FIGURE 9.13 – L'application ouverte dans Edge : « VERSION 2 : Hello from the k8s-web-hello-589649bdcd-5z85f ». (URL : `127.0.0.1:36693`)



FIGURE 9.14 – Après rafraîchissement, un autre pod répond : « … k8s-web-hello-589649bdcd-kmqh4 ». (URL : `127.0.0.1:36693`)

```
P-QGV8NUU:~$ curl -v http://127.0.0.1:36693/; echo
*   Trying 127.0.0.1:36693...
* Connected to 127.0.0.1 (127.0.0.1) port 36693
> GET / HTTP/1.1
> Host: 127.0.0.1:36693
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< X-Powered-By: Express
< Content-Type: text/html; charset=utf-8
< Content-Length: 65
< ETag: W/"41-s35KfGS0rVzIBmqG+WLm+rAuePQ"
< Date: Thu, 15 Oct 2026 22:50:12 GMT
< Connection: keep-alive
< Keep-Alive: timeout=5
<
* Connection #0 to host 127.0.0.1 left intact
<h1>VERSION 2: Hello from the k8s-web-hello-589649bdcd-5z85f</h1>
```

FIGURE 9.15 – Le même tunnel testé depuis le terminal avec `curl -v http://127.0.0.1:36693/` : connexion établie, réponse `HTTP/1.1 200 OK` avec l'en-tête `X-Powered-By: Express` (notre application Node.js) et le corps « VERSION 2 : Hello from the k8s-web-hello-589649bdcd-5z85f ».

```
^Cuser@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ minikube service k8s-web-hello
|-----------|--------------|-------------|---------------------------|
| NAMESPACE |     NAME     | TARGET PORT |            URL            |
|-----------|--------------|-------------|---------------------------|
| default   | k8s-web-hello| 3000        | http://192.168.49.2:32445 |
|-----------|--------------|-------------|---------------------------|
🔗  Starting tunnel for service k8s-web-hello.
|-----------|--------------|-------------|------------------------|
| NAMESPACE |     NAME     | TARGET PORT |          URL           |
|-----------|--------------|-------------|------------------------|
| default   | k8s-web-hello|             | http://127.0.0.1:44999 |
|-----------|--------------|-------------|------------------------|
🎉  Opening service default/k8s-web-hello in default browser...
👉  http://127.0.0.1:44999
❗  Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```

FIGURE 9.16 – Sans `--url`, `minikube service k8s-web-hello` affiche les deux adresses : l'URL du NodePort `http://192.168.49.2:32445` et l'URL du tunnel `http://127.0.0.1:44999`, puis ouvre le navigateur.

Résultat : selon les rafraîchissements, le nom d'hôte affiché change, c'est-à-dire qu'un pod différent a traité la requête. Cela montre la répartition de charge réalisée par le service entre les 4 pods. (La répartition se fait par connexion TCP, pas par requête HTTP : si le navigateur réutilise la même connexion, plusieurs réponses peuvent venir du même pod.)

9.7 Création d'un service LoadBalancer

```
kubectl delete service k8s-web-hello
kubectl expose deployment k8s-web-hello --type=LoadBalancer --port=3000
kubectl get services
minikube service k8s-web-hello
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k delete svc k8s-web-he
llo
service "k8s-web-hello" deleted from default namespace
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k expose deployment k8s
-web-hello --type=LoadBalancer --port=3000
service/k8s-web-hello exposed
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k get svc
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)
  AGE
k8s-web-hello   LoadBalancer   10.99.106.158   <pending>     3000:31186/TCP
  11s
kubernetes      ClusterIP      10.96.0.1       <none>        443/TCP
  5h51m
```

FIGURE 9.17 – Suppression du service NodePort, création d'un service `LoadBalancer` avec `k expose deployment k8s-web-hello --type=LoadBalancer --port=3000`, puis `k get svc` : le service a le ClusterIP `10.99.106.158` et les ports `3000:31186/TCP`, mais la colonne `EXTERNAL-IP` reste à `<pending>`.

Explication : un service LoadBalancer demande une adresse IP externe au fournisseur cloud (AWS, GCP, Azure). Minikube n'est pas un cloud : aucune adresse n'est attribuée et la valeur reste `<pending>`. Un service LoadBalancer contient aussi un NodePort (ici 31186) et un ClusterIP : il reste donc accessible comme en 9.6.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k get svc
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)
  AGE
k8s-web-hello   LoadBalancer   10.99.106.158   <pending>     3000:31186/TCP
  11s
kubernetes      ClusterIP      10.96.0.1       <none>        443/TCP
  5h51m
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ minikube service k8s-we
b-hello
```

NAMESPACE	NAME	TARGET PORT	URL
default	k8s-web-hello	3000	http://192.168.49.2:31186

```
🔗  Starting tunnel for service k8s-web-hello.
```

NAMESPACE	NAME	TARGET PORT	URL
default	k8s-web-hello		http://127.0.0.1:39135

```
🎉  Opening service default/k8s-web-hello in default browser...
```

FIGURE 9.18 – `minikube service k8s-web-hello` affiche l'URL du NodePort (`http://192.168.49.2:31186`) puis, comme on utilise le driver Docker, ouvre un tunnel sur `http://127.0.0.1:39135` et l'ouvre dans le navigateur.



FIGURE 9.19 – L'application répond via le service LoadBalancer : « VERSION 2 : Hello from the k8s-web-hello-589649bdcd-mdrwn ». (URL : `http://127.0.0.1:39135`)

(Autre possibilité : `minikube tunnel`, lancé dans un second terminal, attribue une vraie `EXTERNAL-IP` au service.)

10

Mises à jour progressives (rolling updates) et récupération des pods

> Les captures du navigateur montrent uniquement le contenu de la page ; l'URL utilisée (tunnel `minikube service` sur `127.0.0.1:<port>`, le port change à chaque lancement) est indiquée dans la légende.
>
> À partir d'ici, toutes les sorties sont réelles ; seules les sorties très longues ont été raccourcies : progression de `docker push` (seul l'état final de chaque couche est gardé), fin du `docker build` (`| tail`) et messages « npm notice ».

10.1 Mise à jour progressive du déploiement

La stratégie de mise à jour du déploiement est `RollingUpdate` : lors d'un changement d'image, les nouveaux pods sont créés pendant que les anciens tournent encore, puis les anciens sont supprimés un par un. L'application reste donc disponible pendant toute la mise à jour.

```
k describe deployment k8s-web-hello
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k describe deployment k
8s-web-hello | grep -E "^Name:|Replicas:|StrategyType|RollingUpdateStrategy|
Image:"
Name:                   k8s-web-hello
Replicas:               4 desired | 4 updated | 4 total | 4 available | 0 un
available
StrategyType:           RollingUpdate
RollingUpdateStrategy:  25% max unavailable, 25% max surge
    Image:          hanine169/k8s-web-hello
```

FIGURE 10.1 – `StrategyType: RollingUpdate` avec `RollingUpdateStrategy: 25% max unavailable, 25% max surge` : au maximum 25 % des pods (1 pod sur 4) peuvent être indisponibles, et au maximum 25 % de pods supplémentaires peuvent être créés pendant la mise à jour.

Nouvelle version de l'application. Notre image `latest` renvoyait déjà « VERSION 2 » (modification faite pendant le chapitre 9). Pour que la mise à jour soit visible, on a donc créé une **version 3** : le message de `index.mjs` devient `VERSION 3: Hello from the ...`, et l'image reçoit un nouveau tag, `3.0.0`.

```
docker build . -t hanine169/k8s-web-hello:3.0.0
docker push hanine169/k8s-web-hello:3.0.0
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ grep -n "helloMessage =
" index.mjs
8:  const helloMessage = `<h1>VERSION 3: Hello from the ${os.hostname()}</h1
>`
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ docker build . -t hanin
e169/k8s-web-hello:3.0.0 2>&1 | tail -n 12
#11 exporting manifest sha256:384968a3663dd2bb0ad2eb7c56160e85e1472645331d8c
9af235f4ec25159657 0.0s done
#11 exporting config sha256:8c039d09fc4a5682b8d4b263fe89d6b6c5b5fda9a557e71a
f075e28c4cf5c503 0.0s done
#11 exporting attestation manifest sha256:86e945b9dbddd2766af7753d9c2fb9d95a
6f96ee79111c1dced6a8b5a99be993 0.0s done
#11 exporting manifest list sha256:0e345bdf5c0f9a5f1329ba3873d5e6d476fa10edc
e768842339a13bc8a7fe1fe
#11 exporting manifest list sha256:0e345bdf5c0f9a5f1329ba3873d5e6d476fa10edc
e768842339a13bc8a7fe1fe 0.0s done
#11 naming to docker.io/hanine169/k8s-web-hello:3.0.0 done
#11 unpacking to docker.io/hanine169/k8s-web-hello:3.0.0
#11 unpacking to docker.io/hanine169/k8s-web-hello:3.0.0 0.3s done
#11 DONE 0.9s

 1 warning found (use docker --debug to expand):
 - FromPlatformFlagConstDisallowed: FROM --platform flag should not use cons
tant value "linux/amd64" (line 1)
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ docker images | grep k8
s-web-hello
hanine169/k8s-web-hello:3.0.0
                          0e345bdf5c0f        268MB         65.5MB
hanine169/k8s-web-hello:latest
                          7dc00e85a8dd        268MB         65.5MB
```

FIGURE 10.2 – Le message modifié dans `index.mjs`, la construction de l'image `hanine169/k8s-web-hello:3.0.0`, puis `docker images` : la même image existe maintenant avec deux tags (`latest` et `3.0.0`). L'avertissement `FromPlatformFlagConstDisallowed` vient de la ligne `FROM --platform=linux/amd64` du Dockerfile ; il est sans conséquence.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ docker push hanine169/k
8s-web-hello:3.0.0
The push refers to repository [docker.io/hanine169/k8s-web-hello]
2af57b4eb954: Layer already exists
d428a8ca80cd: Layer already exists
5a84e4ea9a0a: Layer already exists
dd6eb71d102b: Layer already exists
e2de96513ba9: Layer already exists
e2cfdb9ecc2e: Pushed
96afaa34d408: Pushed
ce75b3b95c4c: Layer already exists
3.0.0: digest: sha256:0e345bdf5c0f9a5f1329ba3873d5e6d476fa10edce768842339a13
bc8a7fe1fe size: 856
```

FIGURE 10.3 – `docker push` : la plupart des couches existent déjà sur Docker Hub (`Layer already exists`), seules les couches modifiées sont envoyées (`Pushed`).

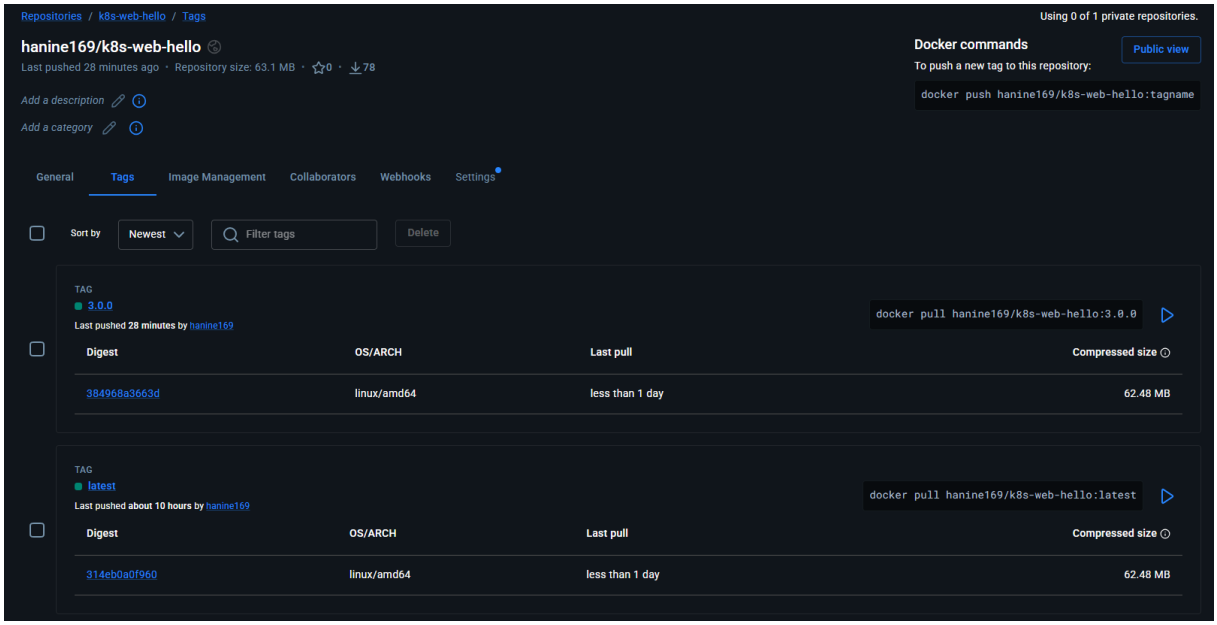


FIGURE 10.4 – Le dépôt `hanine169/k8s-web-hello` sur Docker Hub contient maintenant deux tags : `3.0.0` (poussé à l'instant) et `latest` (poussé environ 10 heures plus tôt). Le digest `384968a3663d` du tag `3.0.0` (linux/amd64) correspond au manifeste affiché à la fin du `docker build` (Figure 10.2).

Mise à jour de l'image du déploiement et suivi du déploiement progressif :

```
k set image deployment k8s-web-hello k8s-web-hello=hanine169/k8s-web-hello:3.0.0
k rollout status deployment k8s-web-hello
k get pods
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k set image deployment
k8s-web-hello k8s-web-hello=hanine169/k8s-web-hello:3.0.0
deployment.apps/k8s-web-hello image updated
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k rollout status deploy
ment k8s-web-hello
Waiting for deployment "k8s-web-hello" rollout to finish: 2 out of 4 new rep
licas have been updated...
Waiting for deployment "k8s-web-hello" rollout to finish: 2 out of 4 new rep
licas have been updated...
Waiting for deployment "k8s-web-hello" rollout to finish: 2 out of 4 new rep
licas have been updated...
Waiting for deployment "k8s-web-hello" rollout to finish: 2 out of 4 new rep
licas have been updated...
Waiting for deployment "k8s-web-hello" rollout to finish: 2 out of 4 new rep
licas have been updated...
Waiting for deployment "k8s-web-hello" rollout to finish: 3 out of 4 new rep
licas have been updated...
Waiting for deployment "k8s-web-hello" rollout to finish: 3 out of 4 new rep
licas have been updated...
Waiting for deployment "k8s-web-hello" rollout to finish: 3 out of 4 new rep
licas have been updated...
Waiting for deployment "k8s-web-hello" rollout to finish: 1 old replicas are
 pending termination...
Waiting for deployment "k8s-web-hello" rollout to finish: 1 old replicas are
 pending termination...
Waiting for deployment "k8s-web-hello" rollout to finish: 1 old replicas are
 pending termination...
Waiting for deployment "k8s-web-hello" rollout to finish: 1 old replicas are
 pending termination...
Waiting for deployment "k8s-web-hello" rollout to finish: 3 of 4 updated rep
licas are available...
Waiting for deployment "k8s-web-hello" rollout to finish: 3 of 4 updated rep
licas are available...
deployment "k8s-web-hello" successfully rolled out
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k get pods
NAME                           READY   STATUS    RESTARTS   AGE
k8s-web-hello-8fffb4549-9dn87   1/1     Running   0          4s
k8s-web-hello-8fffb4549-bfzmn   1/1     Running   0          6s
k8s-web-hello-8fffb4549-cbzmh   1/1     Running   0          12s
k8s-web-hello-8fffb4549-vzt9v   1/1     Running   0          11s
```

FIGURE 10.5 – `k rollout status` montre la progression : « 2 out of 4 new replicas have been updated », puis « 3 out of 4 », « 1 old replicas are pending termination », jusqu'à « successfully rolled out ». `k get pods` affiche 4 nouveaux pods (âge de 4 à 12 s) avec un nouveau hash de ReplicaSet (`8fffb4549`).

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k get svc k8s-web-hello
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)
  AGE
k8s-web-hello   LoadBalancer   10.99.106.158   <pending>     3000:31186/TCP
  9h
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ minikube ssh
docker@minikube:~$ curl -s 10.99.106.158:3000; echo
<h1>VERSION 3: Hello from the k8s-web-hello-8fffb4549-cbzmh</h1>
docker@minikube:~$ curl -s 10.99.106.158:3000; echo
<h1>VERSION 3: Hello from the k8s-web-hello-8fffb4549-vzt9v</h1>
docker@minikube:~$ curl -s 10.99.106.158:3000; echo
<h1>VERSION 3: Hello from the k8s-web-hello-8fffb4549-vzt9v</h1>
docker@minikube:~$ curl -s 10.99.106.158:3000; echo
<h1>VERSION 3: Hello from the k8s-web-hello-8fffb4549-9dn87</h1>
```

FIGURE 10.6 – Depuis le nœud, plusieurs requêtes vers le ClusterIP du service renvoient « VERSION 3 », et proviennent de pods différents.



FIGURE 10.7 – Via `minikube service k8s-web-hello`, le navigateur affiche « VERSION 3 : Hello from the k8s-web-hello-8fffb4549-bfzmn ». (URL : `http://127.0.0.1:43305/`)

Retour en arrière (rollback). Comme dans la vidéo, on revient à l'image sans tag, c'est-à-dire `latest` :

```
k set image deployment k8s-web-hello k8s-web-hello=hanine169/k8s-web-hello
k rollout status deployment k8s-web-hello
```

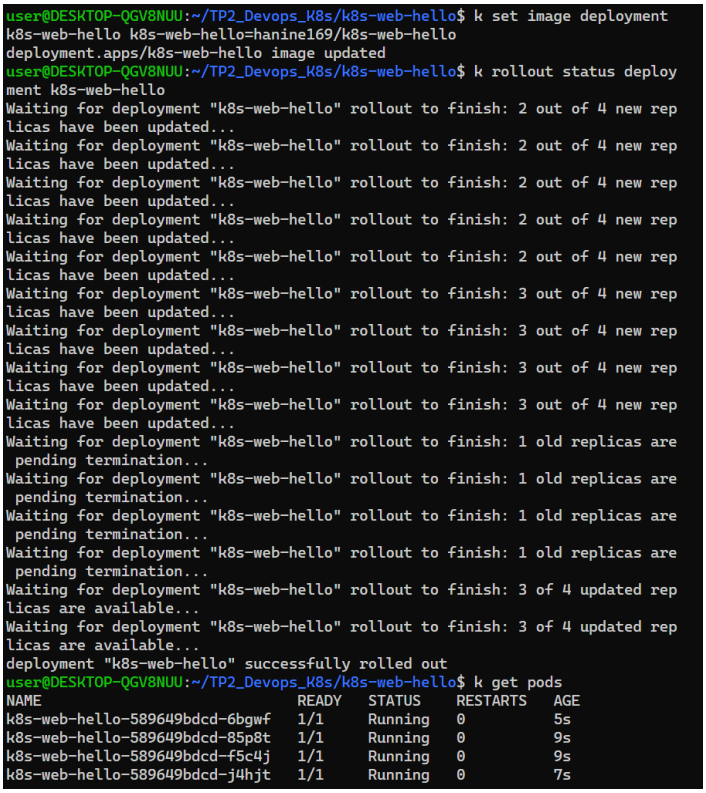


FIGURE 10.8 – Nouveau déploiement progressif vers l'image `latest`. Les nouveaux pods portent à nouveau le hash `589649bdcd` : Kubernetes a **réutilisé l'ancien ReplicaSet** (celui de l'image `latest`) au lieu d'en créer un nouveau.

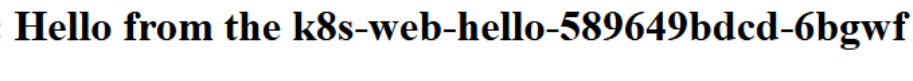


FIGURE 10.9 – Après le rollback, l'application répond de nouveau « VERSION 2 ». (URL : `http://127.0.0.1:41865/`)

> Remarque : on aurait aussi pu revenir à la version précédente avec `k rollout undo deployment k8s-web-hello`.

10.2 Que se passe-t-il quand un pod est supprimé ?

```
k delete pod k8s-web-hello-589649bdcd-6bgwf
k get pods
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k get pods
NAME                             READY   STATUS    RESTARTS   AGE
k8s-web-hello-589649bdcd-6bgwf   1/1     Running   0          30s
k8s-web-hello-589649bdcd-85p8t   1/1     Running   0          34s
k8s-web-hello-589649bdcd-f5c4j   1/1     Running   0          34s
k8s-web-hello-589649bdcd-j4hjt   1/1     Running   0          32s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k delete pod k8s-web-he
llo-589649bdcd-6bgwf
pod "k8s-web-hello-589649bdcd-6bgwf" deleted from default namespace
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ k get pods
NAME                             READY   STATUS              RESTARTS   AGE
k8s-web-hello-589649bdcd-85p8t   1/1     Running             0          37s
k8s-web-hello-589649bdcd-f5c4j   1/1     Running             0          37s
k8s-web-hello-589649bdcd-j4hjt   1/1     Running             0          35s
k8s-web-hello-589649bdcd-r68x2   0/1     ContainerCreating   0          2s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-hello$ sleep 5; k get pods
NAME                             READY   STATUS    RESTARTS   AGE
k8s-web-hello-589649bdcd-85p8t   1/1     Running   0          42s
k8s-web-hello-589649bdcd-f5c4j   1/1     Running   0          42s
k8s-web-hello-589649bdcd-j4hjt   1/1     Running   0          40s
k8s-web-hello-589649bdcd-r68x2   1/1     Running   0          7s
```

FIGURE 10.10 – Après la suppression manuelle du pod `6bgwf`, un nouveau pod `r68x2` apparaît immédiatement à l'état `ContainerCreating` (âge 2 s), puis passe à `Running`. Il y a de nouveau 4 pods.

Explication : le déploiement décrit un **état désiré** (4 réplicas). Le contrôleur du ReplicaSet compare en permanence l'état réel à l'état désiré : dès qu'un pod disparaît, il en crée un autre. C'est le principe d'**auto-réparation** (*self-healing*) de Kubernetes.

11

Tableau de bord Kubernetes (Dashboard)

Pour retrouver l'état de la vidéo (après la mise à jour vers `3.0.0` puis le retour à `latest`), on vérifie d'abord les ReplicaSets, puis on lance le tableau de bord. Avec Minikube, une seule commande suffit :

```
kubectl get rs
minikube dashboard --url
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl set image deployment k8s-web-
hello k8s-web-hello=hanine169/k8s-web-hello
kubectl rollout status deployment k8s-web-hello
deployment "k8s-web-hello" successfully rolled out
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ kubectl get rs
NAME                       DESIRED   CURRENT   READY   AGE
k8s-web-hello-589649bdcd   4         4         4       52s
k8s-web-hello-8fffb4549    0         0         0       35s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ minikube dashboard --url
🤔  Verifying dashboard health ...
🚀  Launching proxy ...
🤔  Verifying proxy health ...
http://127.0.0.1:39575/api/v1/namespaces/kubernetes-dashboard/services/http:
kubernetes-dashboard:/proxy/
```

FIGURE 11.1 – Après le retour à l'image `latest` (`successfully rolled out`), `kubectl get rs` montre les deux ReplicaSets du déploiement : `k8s-web-hello-589649bdcd` (4 pods) et `k8s-web-hello-8fffb4549` (0 pod). `minikube dashboard --url` vérifie l'état du tableau de bord, lance un proxy puis affiche l'URL d'accès `http://127.0.0.1:39575/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/`.

Aucun identifiant n'est demandé, car Minikube est un cluster de test local. Sur un cluster de production, l'accès au tableau de bord doit être sécurisé (jeton, RBAC).

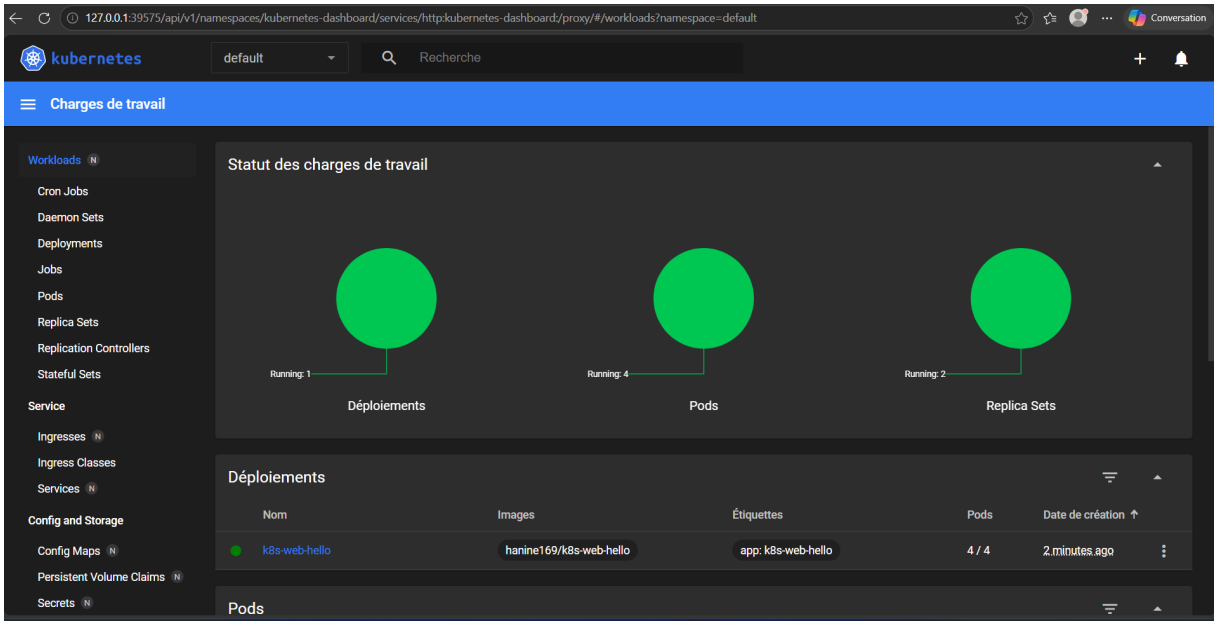


FIGURE 11.2 – La page « Charges de travail » du namespace `default` : 1 déploiement, 4 pods et 2 ReplicaSets à l'état `Running`, et le déploiement `k8s-web-hello` (image `hanine169/k8s-web-hello`, étiquette `app: k8s-web-hello`) avec **4/4** pods. (URL : `127.0.0.1:39575/.../proxy/#/workloads?namespace=default`)

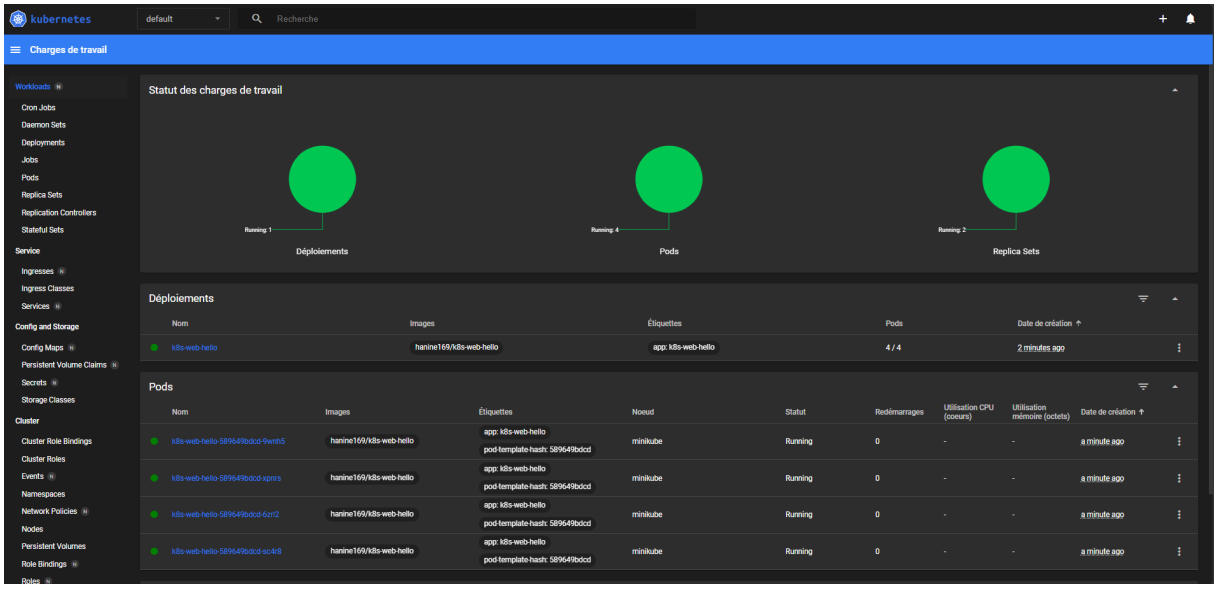


FIGURE 11.3 – Sur la même page, la liste des 4 pods (`9wnh5`, `xpnrs`, `6zrl2`, `sc4r8`) : tous `Running` sur le nœud `minikube`, sans redémarrage, avec les étiquettes `app: k8s-web-hello` et `pod-template-hash: 589649bdcd`.

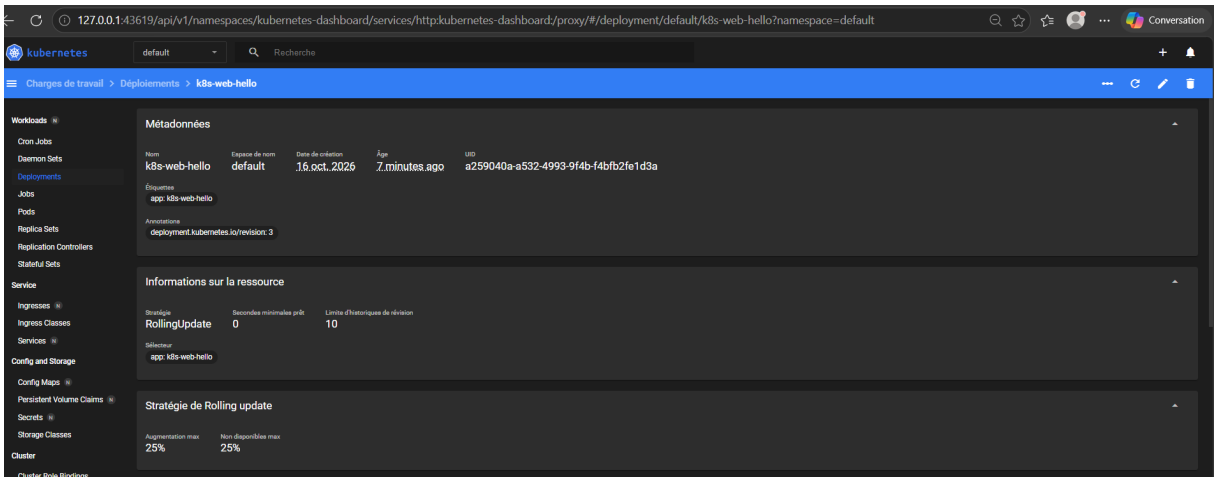


FIGURE 11.4 – Le détail du déploiement : étiquette `app: k8s-web-hello`, annotation `deployment.kubernetes.io/revision: 3` (3 révisions : création, version 3.0.0, puis rollback), stratégie `RollingUpdate` (augmentation max 25 %, non disponibles max 25 %), limite d'historique de 10 révisions et sélecteur `app: k8s-web-hello`. (URL : `127.0.0.1:43619/.../proxy/#/deployment/default/k8s-web-hello`)

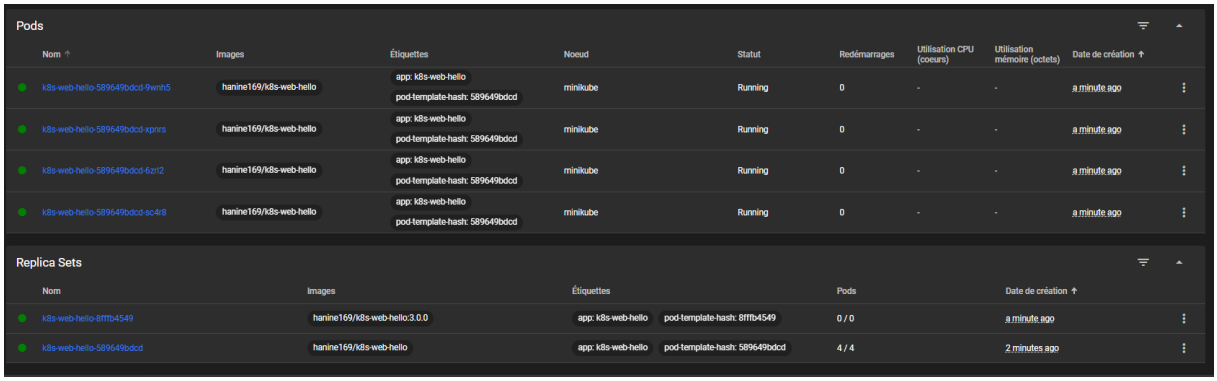


FIGURE 11.5 – Les listes des pods et des ReplicaSets : deux ReplicaSets existent pour ce déploiement, `k8s-web-hello-8fffb4549` (image `3.0.0`, **0/0** pod) et `k8s-web-hello-589649bdcd` (image `latest`, **4/4** pods). C'est la conséquence du rollback : l'ancien ReplicaSet est conservé (à 0 pod) pour permettre un retour en arrière.

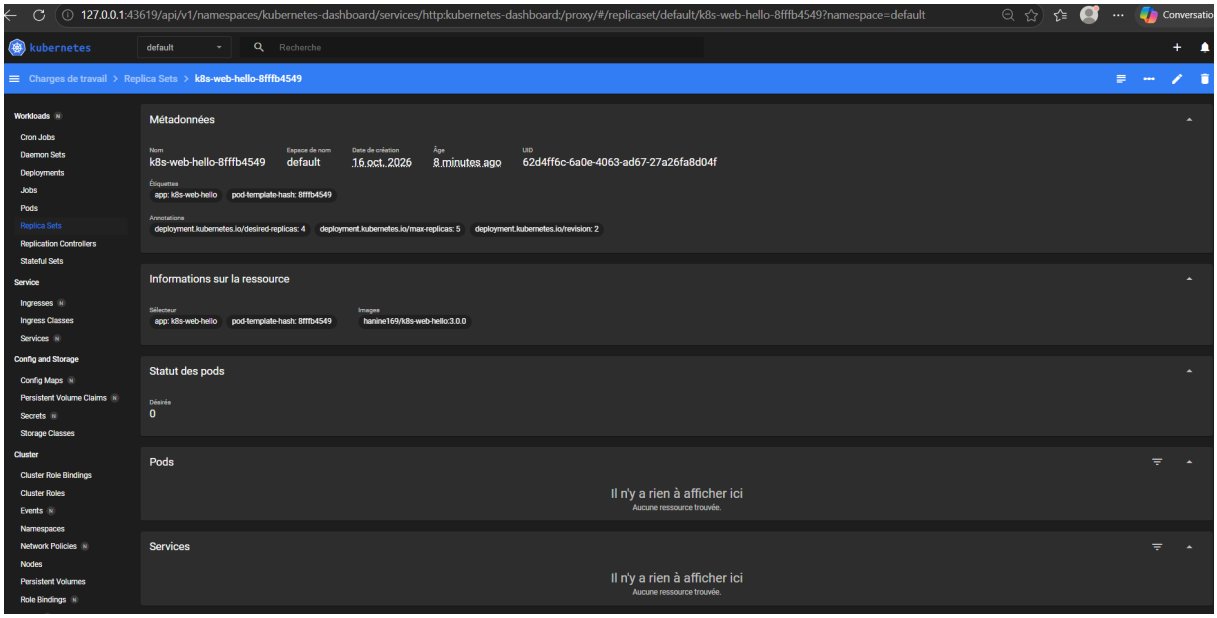


FIGURE 11.6 – Le détail de l'ancien ReplicaSet `k8s-web-hello-8fffb4549` : image `hanine169/k8s-web-hello:3.0.0`, annotation `revision: 2`, **0 pod désiré** et aucun pod (« Il n'y a rien à afficher ici »). Il est gardé vide pour pouvoir revenir à cette version.

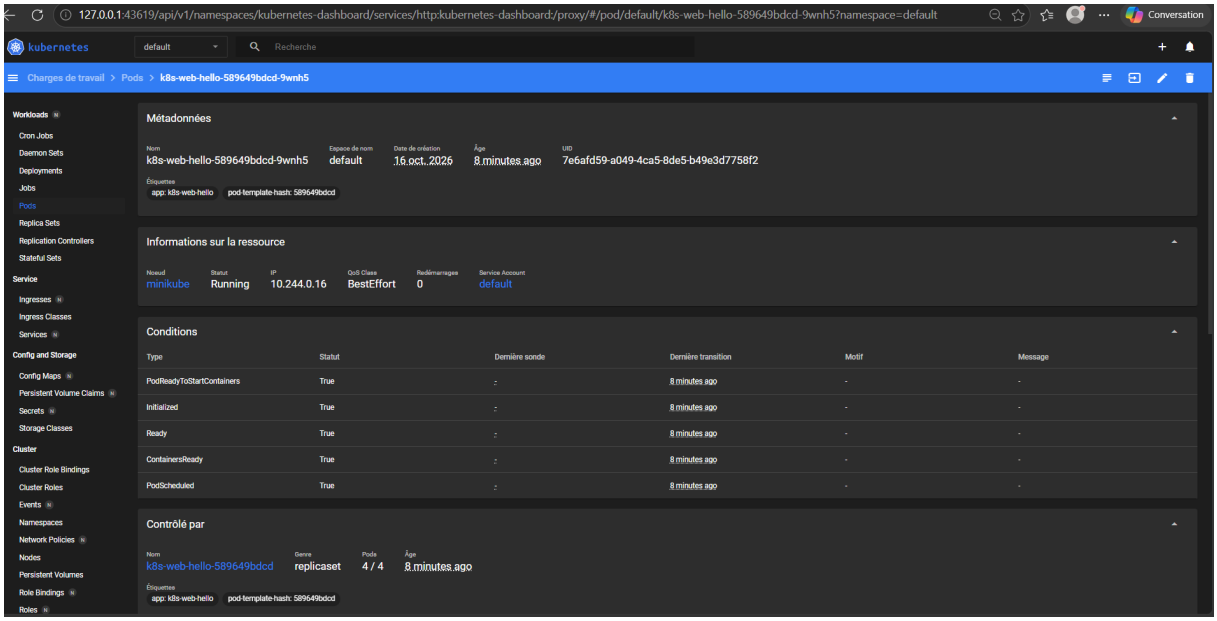


FIGURE 11.7 – Le détail du pod `k8s-web-hello-589649bdcd-9wnh5` : nœud `minikube`, statut `Running`, IP `10.244.0.16`, étiquettes `app: k8s-web-hello` et `pod-template-hash: 589649bdcd`, conditions (`PodReadyToStartContainers`, `Initialized`, `Ready`, `ContainersReady`, `PodScheduled` à `True`), et la section « **Contrôlé par** » : le ReplicaSet `k8s-web-hello-589649bdcd` (4/4 pods). On retrouve la hiérarchie Deployment → ReplicaSet → Pod.

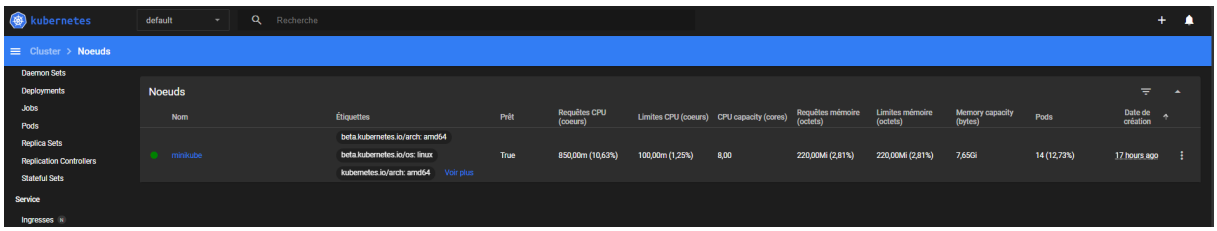


FIGURE 11.8 – La liste des nœuds du cluster : un seul nœud, `minikube`, prêt (`True`), avec 8 cœurs CPU, 7,65 Gi de mémoire et 14 pods en cours d'exécution (sur 110 possibles, soit 12,73 %).

On arrête le tableau de bord avec Ctrl+C dans le terminal.

12

Fichiers de spécification YAML

12.1 Approche impérative vs déclarative

Jusqu'ici, on a utilisé l'**approche impérative** : une commande `kubectl` par action (`create deployment`, `expose`, `scale`, `set image`...). En pratique, on utilise plutôt l'**approche déclarative** : on décrit l'état désiré dans des fichiers YAML, puis on l'applique avec `kubectl apply -f`. Les fichiers peuvent être versionnés (Git), relus et réappliqués à l'identique.

On commence par tout supprimer :

```
k delete all --all
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k delete all --all
pod "k8s-web-hello-589649bdcd-85p8t" deleted from default namespace
pod "k8s-web-hello-589649bdcd-f5c4j" deleted from default namespace
pod "k8s-web-hello-589649bdcd-j4hjt" deleted from default namespace
pod "k8s-web-hello-589649bdcd-r68x2" deleted from default namespace
service "k8s-web-hello" deleted from default namespace
service "kubernetes" deleted from default namespace
deployment.apps "k8s-web-hello" deleted from default namespace
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get pods
NAME                             READY   STATUS        RESTARTS   AGE
k8s-web-hello-589649bdcd-9l87z   0/1     Terminating   0          1s
k8s-web-hello-589649bdcd-d2dcm   0/1     Terminating   0          1s
k8s-web-hello-589649bdcd-dhfm5   0/1     Terminating   0          1s
k8s-web-hello-589649bdcd-xflqw   0/1     Terminating   0          1s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ sleep 35; k get pods
No resources found in default namespace.
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   36s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get deploy
No resources found in default namespace.
```

FIGURE 12.1 – `k delete all --all` supprime les pods, les services et le déploiement du namespace `default`. On remarque que les pods ont été supprimés en premier : le ReplicaSet en a donc recréé 4 aussitôt (âge 1 s), qui passent à `Terminating` dès que le déploiement est supprimé. Le service système `kubernetes` a aussi été supprimé, mais il a été **recréé automatiquement** (âge 36 s).

Fichier `deployment.yaml` (créé dans `~/TP2_Devops_K8s`) :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: k8s-web-hello
spec:
  selector:
    matchLabels:
      app: k8s-web-hello
  template:
    metadata:
      labels:
        app: k8s-web-hello
    spec:
      containers:
      - name: k8s-web-hello
        image: hanine169/k8s-web-hello
        resources:
          limits:
            memory: "128Mi"
            cpu: "500m"
```

```
        ports:
        - containerPort: 3000
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ cat deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: k8s-web-hello
spec:
  selector:
    matchLabels:
      app: k8s-web-hello
  template:
    metadata:
      labels:
        app: k8s-web-hello
    spec:
      containers:
      - name: k8s-web-hello
        image: hanine169/k8s-web-hello
        resources:
          limits:
            memory: "128Mi"
            cpu: "500m"
        ports:
        - containerPort: 3000
```

FIGURE 12.2 – Le contenu du fichier `deployment.yaml`.

Rôle des principaux champs :

— `apiVersion: apps/v1` et `kind: Deployment` : le type d'objet à créer ;

— `metadata.name` : le nom du déploiement ;

— `spec.selector.matchLabels` : l'étiquette qui permet au déploiement de reconnaître **ses** pods ;

— `spec.template` : le **modèle de pod**. Son champ `kind` n'est pas nécessaire, car le modèle d'un déploiement est toujours un pod. Ses `labels` doivent correspondre au `selector` ;

— `containers` : la liste des conteneurs du pod (nom, image, port exposé) ;

— `resources.limits` : les limites de ressources de chaque conteneur : 128 Mio de mémoire et `500m` de CPU, c'est-à-dire la moitié d'un cœur.

12.2 Documentation de l'API Kubernetes

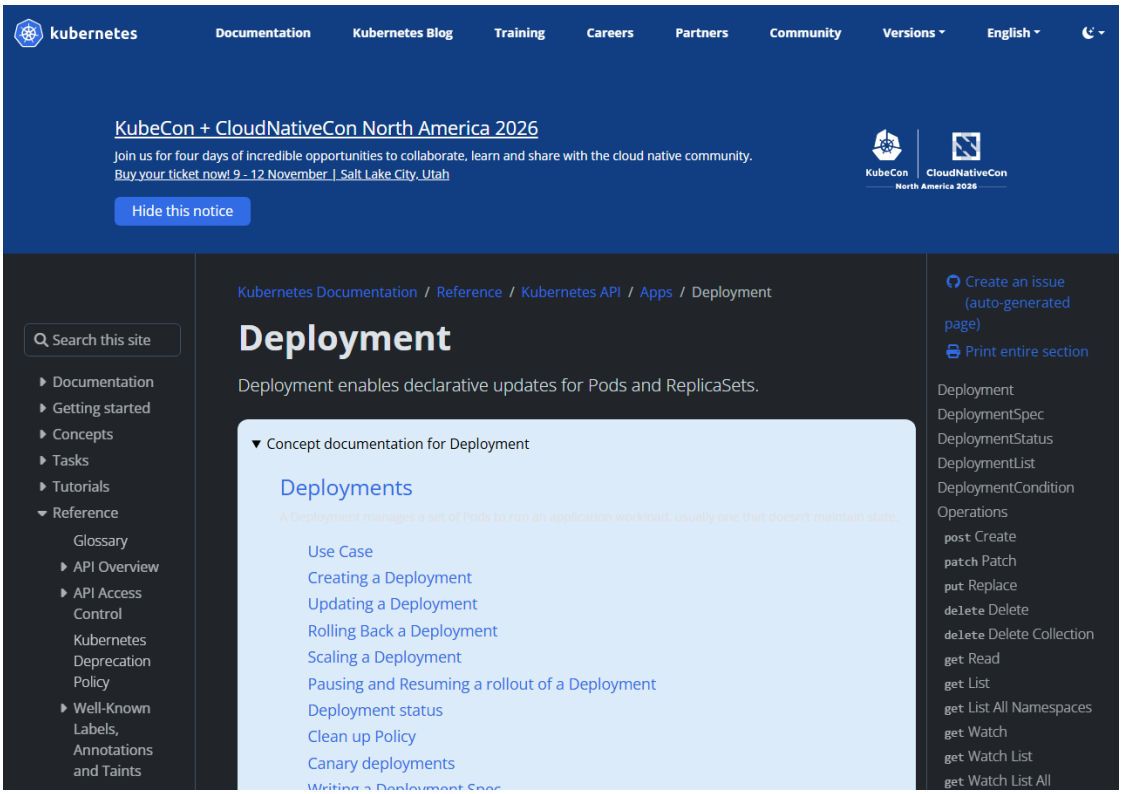


FIGURE 12.3 – La référence de l'API Kubernetes (https://kubernetes.io/docs/reference/kubernetes-api/workload-resources/deployment-v1/) décrit tous les champs d'un Deployment : `apiVersion`, `kind`, `metadata`, `spec` (`DeploymentSpec` : `selector`, `template`, `replicas`, `strategy`...) et `status`. Le champ `status` est rempli par Kubernetes lui-même.

12.3 Application du fichier de déploiement

```
k apply -f deployment.yaml
k get deployments
k get pods
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k apply -f deployment.yaml
deployment.apps/k8s-web-hello created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ sleep 8; k get deployments
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
k8s-web-hello   1/1     1            1           8s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get pods
NAME                             READY   STATUS    RESTARTS   AGE
k8s-web-hello-66888795db-8z6db   1/1     Running   0          8s
```

FIGURE 12.4 – `k apply -f deployment.yaml` crée le déploiement. Comme le champ `replicas` n'est pas indiqué, un seul pod est créé (valeur par défaut : 1).

D'après la documentation, le nombre de pods se règle avec le champ `spec.replicas` (un entier). On ajoute `replicas: 5` et on applique **le même fichier** une seconde fois :

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ head -8 deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: k8s-web-hello
spec:
  replicas: 5
  selector:
    matchLabels:
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k apply -f deployment.yaml
deployment.apps/k8s-web-hello configured
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get pods
NAME                             READY   STATUS              RESTARTS   AGE
k8s-web-hello-66888795db-2xwzw   0/1     ContainerCreating   0          0s
k8s-web-hello-66888795db-8z6db   1/1     Running             0          19s
k8s-web-hello-66888795db-jn5t2   0/1     ContainerCreating   0          0s
k8s-web-hello-66888795db-nsmqv   0/1     ContainerCreating   0          0s
k8s-web-hello-66888795db-zl5t5   0/1     ContainerCreating   0          0s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ sleep 6; k get pods
NAME                             READY   STATUS    RESTARTS   AGE
k8s-web-hello-66888795db-2xwzw   1/1     Running   0          6s
k8s-web-hello-66888795db-8z6db   1/1     Running   0          25s
k8s-web-hello-66888795db-jn5t2   1/1     Running   0          6s
k8s-web-hello-66888795db-nsmqv   1/1     Running   0          6s
k8s-web-hello-66888795db-zl5t5   1/1     Running   0          6s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   71s
```

FIGURE 12.5 – Après l'ajout de `replicas: 5`, `k apply` indique `configured` (et non `created`) : Kubernetes compare le fichier à l'état actuel et n'applique que la différence. 4 nouveaux pods sont créés et passent à `Running` en quelques secondes. Aucun service n'existe encore.

12.4 Fichier de service

Fichier `service.yaml` :

```
apiVersion: v1
```

```
kind: Service
metadata:
  name: k8s-web-hello
spec:
  type: LoadBalancer
  selector:
    app: k8s-web-hello
  ports:
  - port: 3030
    targetPort: 3000
```

— `apiVersion: v1` (différent de `apps/v1` pour un déploiement) ;

— `selector` : les pods ciblés, ceux qui portent l'étiquette `app: k8s-web-hello` ;

— `port: 3030` : le port du service ; `targetPort: 3000` : le port du conteneur (Express) ;

— `type: LoadBalancer` : sans ce champ, le type serait `ClusterIP` (valeur par défaut). Les autres valeurs possibles sont `NodePort` et `ExternalName`.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: k8s-web-hello
spec:
  type: LoadBalancer
  selector:
    app: k8s-web-hello
  ports:
  - port: 3030
    targetPort: 3000
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k apply -f service.yaml
service/k8s-web-hello created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get svc
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)
  AGE
k8s-web-hello   LoadBalancer   10.104.108.68   <pending>     3030:30432/TCP
  0s
kubernetes      ClusterIP      10.96.0.1       <none>        443/TCP
  79s
```

FIGURE 12.6 – Le fichier `service.yaml`, son application avec `k apply -f service.yaml`, et le service créé : type `LoadBalancer`, ports `3030:30432/TCP`, `EXTERNAL-IP` à `<pending>`.

VERSION 2: Hello from the k8s-web-hello-66888795db-jn5t2

(a)

VERSION 2: Hello from the k8s-web-hello-66888795db-zl5t5

(b)

FIGURE 12.7 – Via `minikube service k8s-web-hello`, deux requêtes successives obtiennent la réponse de deux pods différents (`jn5t2` puis `zl5t5`) : tout fonctionne comme avant, mais l'ensemble a été créé de façon déclarative. (URL : `http://127.0.0.1:44381/`)

13

Création de plusieurs déploiements

13.1 Suppression via les fichiers YAML et architecture cible

Les ressources créées à partir de fichiers peuvent être supprimées à partir de ces mêmes fichiers :

```
k delete -f deployment.yaml -f service.yaml
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k delete -f deployment.yaml -f servic
e.yaml
deployment.apps "k8s-web-hello" deleted from default namespace
service "k8s-web-hello" deleted from default namespace
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   2m10s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get deploy
No resources found in default namespace.
```

FIGURE 13.1 – Le déploiement et le service sont supprimés en une commande ; il ne reste que le service `kubernetes`.

Objectif : faire communiquer **deux déploiements**, ce qui est très courant (frontend → backend, backend → base de données...).

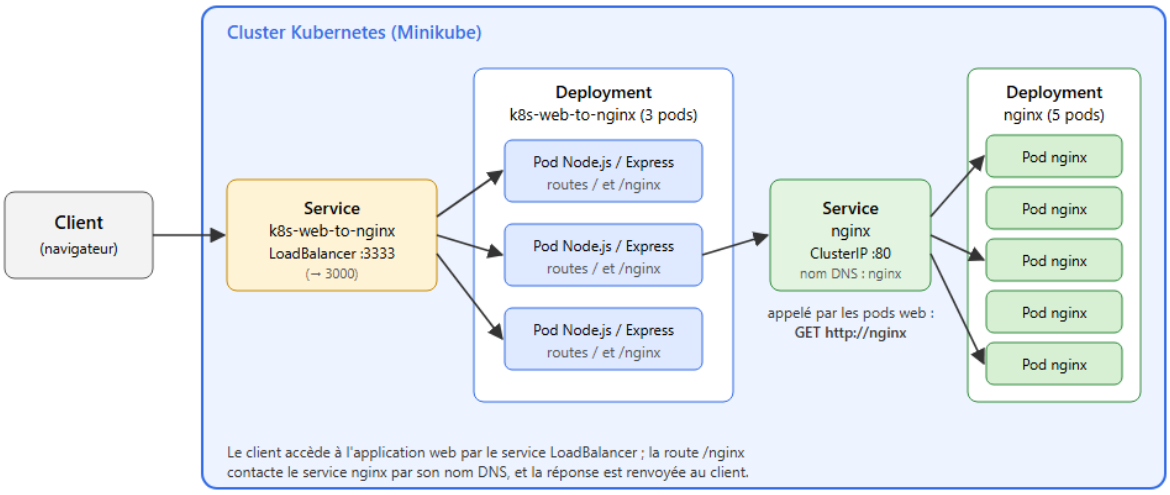


FIGURE 13.2 – Schéma de l'architecture : le client accède à l'application web (`k8s-web-to-nginx`, 3 pods) par un service **LoadBalancer**. La route `/nginx` de l'application envoie une requête au service **ClusterIP** `nginx`, qui répartit la charge entre les 5 pods nginx. La réponse est ensuite renvoyée au client.

13.2 Application `k8s-web-to-nginx`

On copie le dossier `k8s-web-hello` en `k8s-web-to-nginx` (le Dockerfile reste identique) et on modifie `index.mjs` :

```
import express from 'express'
import fetch from 'node-fetch'
import os from 'os'

const app = express()
const PORT = 3000

app.get("/", (req, res) => {
  const helloMessage = `Hello from the ${os.hostname()}`
  console.log(helloMessage)
  res.send(helloMessage)
})

app.get("/nginx", async (req, res) => {
  const url = 'http://nginx'
  const response = await fetch(url)
  const body = await response.text()
  res.send(body)
})
```

```
app.listen(PORT, () => {
  console.log(`Web server is listening at port ${PORT}`)
})
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-to-nginx$ cat index.mjs
import express from 'express'
import fetch from 'node-fetch'
import os from 'os'

const app = express()
const PORT = 3000

app.get("/", (req, res) => {
  const helloMessage = `Hello from the ${os.hostname()}`
  console.log(helloMessage)
  res.send(helloMessage)
})

app.get("/nginx", async (req, res) => {
  const url = 'http://nginx'
  const response = await fetch(url)
  const body = await response.text()
  res.send(body)
})

app.listen(PORT, () => {
  console.log(`Web server is listening at port ${PORT}`)
})
```

FIGURE 13.3 – Le nouveau fichier `index.mjs`. La route `/nginx` est une fonction asynchrone qui envoie une requête à `http://nginx`, attend la réponse et la renvoie au client : elle sert de **proxy**. L'URL utilise le **nom du service** (`nginx`) et non son ClusterIP : le ClusterIP est attribué dynamiquement, alors que le nom du service est fixe.

```
npm install node-fetch
rm -rf node_modules
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-to-nginx$ npm install node-fet
ch

added 6 packages, and audited 72 packages in 6s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOME
xception instead
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-to-nginx$ cat package.json
{
  "name": "k8s-web-to-nginx",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.mjs"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.3.0",
    "node-fetch": "^3.3.2"
  }
}
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-to-nginx$ rm -rf node_modules
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-to-nginx$ ls
Dockerfile
index.mjs
package-lock.json
package.json
```

FIGURE 13.4 – Installation du paquet `node-fetch` : il est ajouté aux dépendances dans `package.json` (avec `express`). Le dossier `node_modules` est ensuite supprimé, car `npm install` est exécuté pendant la construction de l'image (instruction `RUN npm install` du Dockerfile).

13.3 Construction et publication de l'image

```
docker build . -t hanine169/k8s-web-to-nginx
docker push hanine169/k8s-web-to-nginx
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-to-nginx$ docker build . -t ha
nine169/k8s-web-to-nginx 2>&1 | grep -E "^#[0-9]+ \[|npm install|DONE|naming
" | tail -n 14
#5 [1/5] FROM docker.io/library/node:alpine@sha256:6f08a523443ddedf52941874c
203475b0ad503e859ad70cb26ab8be9a517dbd5
#5 DONE 0.0s
#6 [internal] load build context
#6 DONE 0.0s
#7 [2/5] WORKDIR /app
#8 [3/5] COPY package.json package-lock.json ./
#8 DONE 0.0s
#9 [4/5] RUN npm install
#9 6.835 npm notice To update run: npm install -g npm@12.2.0
#9 DONE 7.2s
#10 [5/5] COPY . ./
#10 DONE 0.3s
#11 naming to docker.io/hanine169/k8s-web-to-nginx:latest done
#11 DONE 2.3s
```

FIGURE 13.5 – Les 5 étapes du Dockerfile (`FROM`, `WORKDIR`, `COPY`, `RUN npm install`, `COPY`) et l'image nommée `hanine169/k8s-web-to-nginx:latest` (tag `latest` ajouté automatiquement).

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/k8s-web-to-nginx$ docker push hanine16
9/k8s-web-to-nginx
Using default tag: latest
The push refers to repository [docker.io/hanine169/k8s-web-to-nginx]
d4e4652c2aa5: Pushed
dd6eb71d102b: Mounted from hanine169/k8s-web-hello
e2de96513ba9: Mounted from hanine169/k8s-web-hello
ce75b3b95c4c: Mounted from hanine169/k8s-web-hello
2af57b4eb954: Mounted from hanine169/k8s-web-hello
7c052b61d6a5: Pushed
911484408d97: Pushed
731054ae7444: Pushed
latest: digest: sha256:f747f756a63339034ad50c3f75753760ab3a7b6e100ae3acceb95
998dac124a5 size: 856
```

FIGURE 13.6 – `docker push` : plusieurs couches sont « Mounted from hanine169/k8s-web-hello », c'est-à-dire réutilisées depuis l'autre image sur Docker Hub au lieu d'être renvoyées.

13.4 Fichier combiné service + déploiement

Un seul fichier YAML peut contenir plusieurs objets, séparés par une ligne `---`. Fichier `k8s-web-to-nginx.yaml` :

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ cat k8s-web-to-nginx.yaml
apiVersion: v1
kind: Service
metadata:
  name: k8s-web-to-nginx
spec:
  type: LoadBalancer
  selector:
    app: k8s-web-to-nginx
  ports:
  - port: 3333
    targetPort: 3000
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: k8s-web-to-nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: k8s-web-to-nginx
  template:
    metadata:
      labels:
        app: k8s-web-to-nginx
    spec:
      containers:
      - name: k8s-web-to-nginx
        image: hanine169/k8s-web-to-nginx
        resources:
          limits:
            memory: "128Mi"
            cpu: "500m"
        ports:
        - containerPort: 3000
```

FIGURE 13.7 – Dans un même fichier : le **service** `k8s-web-to-nginx` (type `LoadBalancer`, port `3333` → `targetPort 3000`), le séparateur `---`, puis le **déploiement** `k8s-web-to-nginx` (3 réplicas, image `hanine169/k8s-web-to-nginx`, port 3000).

13.5 Déploiement nginx

Fichier `nginx.yaml` :

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ cat nginx.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  selector:
    app: nginx
  ports:
  - port: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
        resources:
          limits:
            memory: "128Mi"
            cpu: "500m"
        ports:
        - containerPort: 80
```

FIGURE 13.8 – Le service `nginx` sans champ `type`, donc de type `ClusterIP` (accessible uniquement depuis l'intérieur du cluster). Il n'a qu'un `port: 80` : sans `targetPort`, le port cible est le même (80). Le déploiement `nginx` utilise l'image officielle `nginx`, avec 5 réplicas et le port 80. **Le nom du service doit être exactement `nginx`**, car c'est le nom utilisé dans `index.mjs` (`http://nginx`).

13.6 Application des deux fichiers

```
k apply -f k8s-web-to-nginx.yaml -f nginx.yaml
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get deploy
No resources found in default namespace.
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   4m17s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ ls *.yaml
deployment.yaml
k8s-web-to-nginx.yaml
nginx.yaml
service.yaml
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k apply -f k8s-web-to-nginx.yaml -f n
ginx.yaml
service/k8s-web-to-nginx created
deployment.apps/k8s-web-to-nginx created
service/nginx created
deployment.apps/nginx created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get deploy
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
k8s-web-to-nginx   0/3     3            0           2s
nginx              0/5     5            0           2s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get svc
NAME               TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)
     AGE
k8s-web-to-nginx   LoadBalancer   10.110.77.47    <pending>     3333:31706/T
CP   3s
kubernetes         ClusterIP      10.96.0.1       <none>        443/TCP
     4m21s
nginx              ClusterIP      10.107.232.55   <none>        80/TCP
     3s
```

FIGURE 13.9 – Au départ, aucun déploiement. Une seule commande crée 2 services et 2 déploiements. `k get svc` montre `k8s-web-to-nginx` (LoadBalancer, `3333:31706/TCP`) et `nginx` (ClusterIP `10.107.232.55`, `80/TCP`).

14

Interaction entre les déploiements

14.1 Test de la communication

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ sleep 15; k get pods
NAME                                READY   STATUS    RESTARTS   AGE
k8s-web-to-nginx-6b884cf46d-p8fj9   1/1     Running   0          30s
k8s-web-to-nginx-6b884cf46d-xzr4s   1/1     Running   0          30s
k8s-web-to-nginx-6b884cf46d-znxv9   1/1     Running   0          30s
nginx-8d87469cc-cnwfw               1/1     Running   0          30s
nginx-8d87469cc-lnbd5               1/1     Running   0          30s
nginx-8d87469cc-rqpv7               1/1     Running   0          30s
nginx-8d87469cc-t8pqd               1/1     Running   0          30s
nginx-8d87469cc-v2drr               1/1     Running   0          30s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get deploy
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
k8s-web-to-nginx  3/3     3            3           31s
nginx             5/5     5            5           31s
```

FIGURE 14.1 – Les 3 pods `k8s-web-to-nginx` et les 5 pods `nginx` sont `Running` ; les deux déploiements sont prêts (3/3 et 5/5).

On ouvre l'application avec `minikube service k8s-web-to-nginx` :

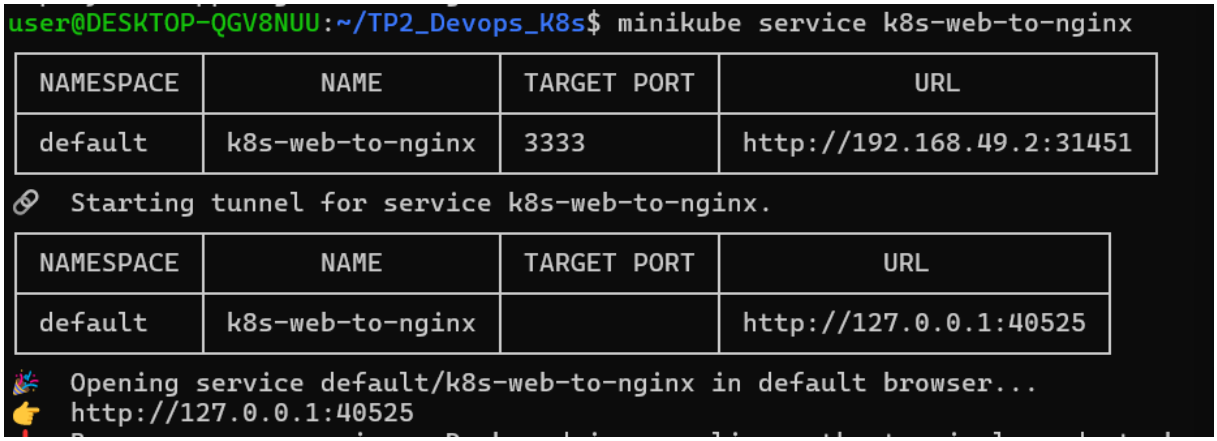


FIGURE 14.2 – `minikube service k8s-web-to-nginx` affiche l'URL du NodePort (`http://192.168.49.2:31451`, port de service 3333) puis, comme on utilise le driver Docker, ouvre un tunnel sur `http://127.0.0.1:40525` dans le navigateur.

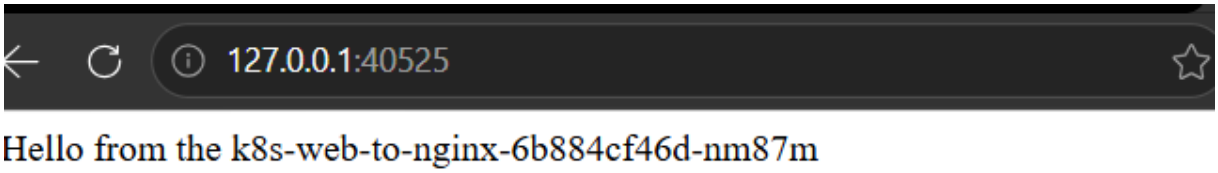


FIGURE 14.3 – La route `/` répond « Hello from the k8s-web-to-nginx-6b884cf46d-nm87m » : la réponse vient d'un des pods du déploiement web. (Le déploiement a été réappliqué pour ces captures du navigateur, d'où des noms de pods différents de ceux de la Figure 14.1.) (URL : `http://127.0.0.1:40525`)

> Remarque : en rafraîchissant la page, le nom du pod ne change pas. Ce n'est pas une erreur : le service répartit la charge **par connexion TCP**, pas par requête HTTP, et le navigateur réutilise la même connexion (*keep-alive*). Toutes les requêtes passent donc par le même pod. Avec plusieurs requêtes `curl` (une nouvelle connexion à chaque fois), on voit bien des pods différents (voir Figures 9.8 et 10.6).

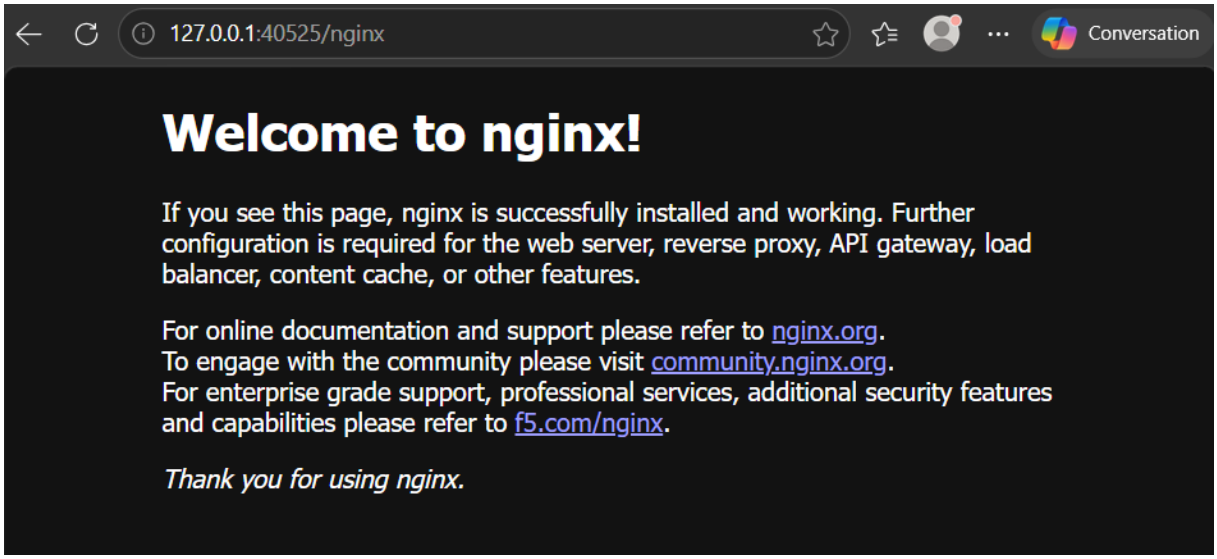


FIGURE 14.4 – La route `/nginx` affiche « Welcome to nginx ! ». La requête a atteint un pod web, qui a contacté le service `nginx` **par son nom**, puis renvoyé la page reçue d'un des pods nginx. Les deux déploiements communiquent donc entre eux. (URL : `http://127.0.0.1:40525/nginx`)

14.2 Résolution DNS du nom du service

La traduction du nom `nginx` en adresse IP est faite par le **DNS interne** de Kubernetes (CoreDNS). On peut le vérifier en exécutant une commande dans un pod avec `kubectl exec`, l'équivalent de `docker exec` :

```
k exec k8s-web-to-nginx-6b884cf46d-xzr4s -- nslookup nginx
k get svc nginx
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get pods -l app=k8s-web-to-nginx
NAME                                READY   STATUS    RESTARTS   AGE
k8s-web-to-nginx-6b884cf46d-p8fj9   1/1     Running   0          73s
k8s-web-to-nginx-6b884cf46d-xzr4s   1/1     Running   0          73s
k8s-web-to-nginx-6b884cf46d-znxv9   1/1     Running   0          73s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k exec k8s-web-to-nginx-6b884cf46d-xz
r4s -- nslookup nginx
Server:         10.96.0.10
Address:        10.96.0.10:53


** server can't find nginx.svc.cluster.local: NXDOMAIN

** server can't find nginx.svc.cluster.local: NXDOMAIN

** server can't find nginx.cluster.local: NXDOMAIN

** server can't find nginx.cluster.local: NXDOMAIN

Name:   nginx.default.svc.cluster.local
Address: 10.107.232.55

command terminated with exit code 1
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get svc nginx
NAME    TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
nginx   ClusterIP   10.107.232.55   <none>        80/TCP    75s
```

FIGURE 14.5 – Le serveur DNS utilisé est `10.96.0.10` (le service CoreDNS). Le nom `nginx` est résolu en `nginx.default.svc.cluster.local` → **`10.107.232.55`**, exactement le ClusterIP du service `nginx` affiché par `k get svc nginx`.

> Les lignes `NXDOMAIN` et le message `exit code 1` sont normaux : `nslookup` (BusyBox, image Alpine) essaie aussi les autres domaines de recherche du pod (`svc.cluster.local`, `cluster.local`), qui n'existent pas, avant de trouver le nom complet `nginx.default.svc.cluster.local`. Le nom complet d'un service est `<service>.<namespace>.svc.cluster.local`.

```
k exec k8s-web-to-nginx-6b884cf46d-xzr4s -- wget -qO- http://nginx
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k exec k8s-web-to-nginx-6b884cf46d-xz
r4s -- wget -qO- http://nginx
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

FIGURE 14.6 – Depuis un pod web, `wget` récupère directement le code HTML de la page d'accueil d'un pod nginx en utilisant uniquement le nom du service.

14.3 Suppression

```
k delete -f nginx.yaml -f k8s-web-to-nginx.yaml
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k delete -f nginx.yaml -f k8s-web-to-
nginx.yaml
service "nginx" deleted from default namespace
deployment.apps "nginx" deleted from default namespace
service "k8s-web-to-nginx" deleted from default namespace
deployment.apps "k8s-web-to-nginx" deleted from default namespace
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get deploy
No resources found in default namespace.
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   5m58s
```

FIGURE 14.7 – Les deux services et les deux déploiements sont supprimés en une commande ; il ne reste que le service `kubernetes`.

Conclusion : le déploiement avec des fichiers YAML est plus rapide, reproductible et lisible que l'enchaînement de commandes `kubectl` séparées.

15

Le runtime de conteneurs CRI-O

15.1 Changement de runtime

Kubernetes n'exécute pas les conteneurs lui-même : sur chaque nœud, le **kubelet** pilote un **runtime de conteneurs** via une interface standard, la **CRI** (*Container Runtime Interface*). Plusieurs runtimes compatibles existent : containerd, CRI-O, ou Docker Engine via l'adaptateur `cri-dockerd`. **CRI-O** est un runtime léger, conçu uniquement pour Kubernetes (c'est le runtime par défaut d'OpenShift).

Dans la vidéo, le formateur supprime son cluster et le recrée avec `--container-runtime=cri-o`. Il précise que cela n'a pas fonctionné pour lui avec le driver Docker et utilise VirtualBox.

Adaptation : pour ne pas supprimer notre cluster principal, on a créé un **second cluster** (profil Minikube `crio`), avec le driver Docker :

```
minikube start -p crio --driver=docker --container-runtime=cri-o --memory=2200
minikube -p crio status
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ minikube start -p crio --driver=docke
r --container-runtime=cri-o --memory=2200
* [crio] minikube v1.39.0 on Ubuntu 24.04 (kvm/amd64)
* Using the docker driver based on user configuration
* Using Docker driver with root privileges
* Starting "crio" primary control-plane node in "crio" cluster
* Pulling base image v0.0.51 ...
* Downloading Kubernetes v1.37.0 preload ...
* Preparing Kubernetes v1.37.0 on CRI-O 1.35.7 ...
* Configuring CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: default-storageclass, storage-provisioner
* Done! kubectl is now configured to use "crio" cluster and "default" namesp
ace by default
! For an improved experience it's recommended to use Docker Engine instead o
f Docker Desktop.
Docker Engine installation instructions: https://docs.docker.com/engine/inst
all/#server
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ minikube -p crio status
crio
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

FIGURE 15.1 – Création du cluster `crio` : « Preparing Kubernetes v1.37.0 on **CRI-O 1.35.7** ». Avec une version récente de Minikube, le driver Docker fonctionne bien avec CRI-O. Le statut indique que le host, le kubelet et l'apiserver sont `Running`.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ minikube -p crio ssh
docker@crio:~$ docker ps
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the d
ocker daemon running?
ssh: Process exited with status 1
docker@crio:~$ sudo crictl version
Version:  0.1.0
RuntimeName:  cri-o
RuntimeVersion:  1.35.7
RuntimeApiVersion:  v1
docker@crio:~$ sudo crictl ps | awk 'NR==1{printf "%-15s %-9s %-25s %s\n","C
ONTAINER","STATE","NAME","NAMESPACE"} NR>1{printf "%-15s %-9s %-25s %s\n", $
1, $(NF-5), $(NF-4), $NF}'
CONTAINER       STATE     NAME                      NAMESPACE
8570ee60856d4   Running   coredns                   kube-system
8e273df07207a   Running   storage-provisioner       kube-system
e3f9a775ba491   Running   kindnet-cni               kube-system
7566fa4ee144f   Running   kube-proxy                kube-system
3188a885a396a   Running   kube-scheduler            kube-system
7ebdfc584ca91   Running   etcd                      kube-system
86c02f66bb181   Running   kube-controller-manager   kube-system
562c98f31ae4d   Running   kube-apiserver            kube-system
```

FIGURE 15.2 – Dans le nœud (`minikube -p crio ssh`) : `docker ps` échoue (« Cannot connect to the Docker daemon »), car Docker ne tourne pas dans ce nœud. `sudo crictl version` indique le runtime `cri-o 1.35.7`, et `sudo crictl ps` liste les conteneurs du cluster (kube-apiserver, etcd, kube-scheduler, kube-proxy, coredns…) gérés par CRI-O. (Les colonnes de `crictl ps` ont été filtrées avec `awk` pour tenir dans la largeur du terminal.)

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get nodes -o custom-columns=NAME:.m
etadata.name,RUNTIME:.status.nodeInfo.containerRuntimeVersion
NAME   RUNTIME
crio   cri-o://1.35.7
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k --context minikube get nodes -o cus
tom-columns=NAME:.metadata.name,RUNTIME:.status.nodeInfo.containerRuntimeVer
sion
NAME       RUNTIME
minikube   containerd://2.3.4
```

FIGURE 15.3 – Comparaison des deux clusters : le nœud `crio` utilise `cri-o://1.35.7`, et notre cluster principal `minikube` utilise `containerd://2.3.4`.

> Remarque : notre cluster principal n'a jamais utilisé Docker comme runtime (voir section 4.2) : c'est pour cela qu'on utilise `crictl` au lieu de `docker ps` depuis le début du TP.

15.2 Déploiement des applications sur CRI-O

On applique **exactement les mêmes fichiers YAML** que dans le chapitre 13 :

```
k apply -f k8s-web-to-nginx.yaml -f nginx.yaml
```

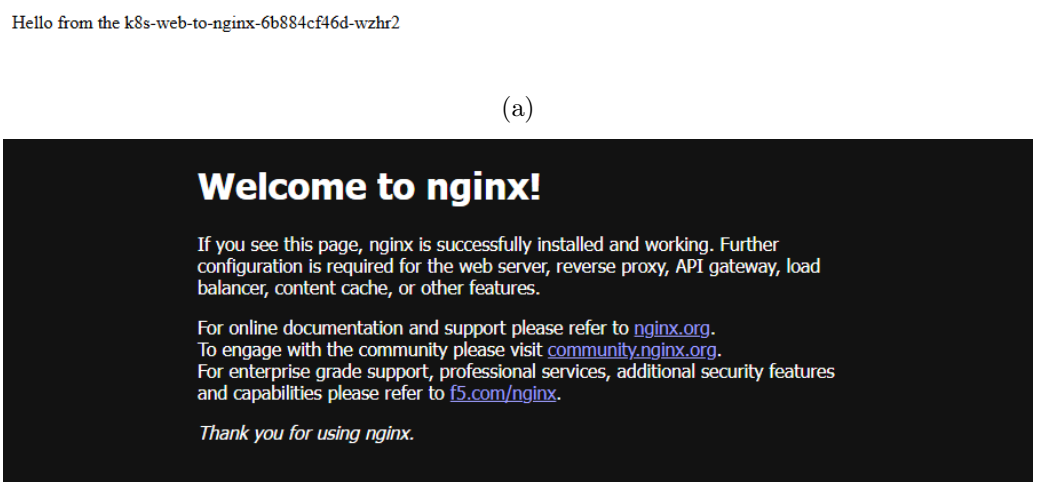
```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k config current-context
crio
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k apply -f k8s-web-to-nginx.yaml -f n
ginx.yaml
service/k8s-web-to-nginx created
deployment.apps/k8s-web-to-nginx created
service/nginx created
deployment.apps/nginx created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get deploy
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
k8s-web-to-nginx   0/3     3            0           1s
nginx             0/5     5            0           1s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get svc
NAME              TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)
     AGE
k8s-web-to-nginx   LoadBalancer   10.104.131.69   <pending>     3333:30938/T
CP   2s
kubernetes        ClusterIP      10.96.0.1       <none>        443/TCP
     2m6s
nginx             ClusterIP      10.109.63.231   <none>        80/TCP
     2s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ sleep 40; k get pods
NAME                                READY   STATUS              RESTARTS   A
GE
k8s-web-to-nginx-6b884cf46d-nskjn   0/1     ContainerCreating   0          4
3s
k8s-web-to-nginx-6b884cf46d-wzhr2   0/1     ContainerCreating   0          4
3s
k8s-web-to-nginx-6b884cf46d-xj5zf   0/1     ContainerCreating   0          4
3s
nginx-8d87469cc-9cqxp               1/1     Running             0          4
3s
nginx-8d87469cc-c9lq8               0/1     ContainerCreating   0          4
3s
nginx-8d87469cc-npndr               1/1     Running             0          4
3s
nginx-8d87469cc-qssg4               0/1     ContainerCreating   0          4
3s
nginx-8d87469cc-qswxj               0/1     ContainerCreating   0          4
3s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get deploy
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
k8s-web-to-nginx   0/3     3            0           44s
nginx             2/5     5            2           44s
```

FIGURE 15.4 – Le contexte courant est `crio`. Les 2 services et les 2 déploiements sont créés. Après 40 s, une partie des pods est encore en `ContainerCreating`, car le nouveau cluster doit retélécharger toutes les images.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ sleep 60; k get pods
NAME                                READY   STATUS    RESTARTS   AGE
k8s-web-to-nginx-6b884cf46d-nskjn   1/1     Running   0          2m
k8s-web-to-nginx-6b884cf46d-wzhr2   1/1     Running   0          2m
k8s-web-to-nginx-6b884cf46d-xj5zf   1/1     Running   0          2m
nginx-8d87469cc-9cqxp               1/1     Running   0          2m
nginx-8d87469cc-c9lq8               1/1     Running   0          2m
nginx-8d87469cc-npndr               1/1     Running   0          2m
nginx-8d87469cc-qssg4               1/1     Running   0          2m
nginx-8d87469cc-qswxj               1/1     Running   0          2m
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get deploy
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
k8s-web-to-nginx   3/3     3            3           2m1s
nginx             5/5     5            5           2m1s
```

FIGURE 15.5 – Après environ 2 minutes, les 8 pods sont `Running` et les deux déploiements sont prêts (3/3 et 5/5).

15.3 Vérification



(a)

(b)

FIGURE 15.6 – Via `minikube -p crio service k8s-web-to-nginx` (tunnel `http://127.0.0.1:45859` pour `/`, `http://127.0.0.1:46469` pour `/nginx`), la route `/` répond « Hello from the k8s-web-to-nginx-6b884cf46d-wzhr2 », et `/nginx` affiche « Welcome to nginx ! » : l'application fonctionne exactement comme avec containerd.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ minikube -p crio ssh
docker@crio:~$ sudo crictl ps | awk 'NR==1{printf "%-15s %-9s %-25s %s\n","C
ONTAINER","STATE","NAME","NAMESPACE"} NR>1{printf "%-15s %-9s %-25s %s\n", $
1, $(NF-5), $(NF-4), $NF}' | grep -E 'NAME|k8s-web|nginx'
CONTAINER       STATE     NAME                      NAMESPACE
d4014c22e594f   Running   k8s-web-to-nginx          default
fca0be466db0d   Running   nginx                     default
eb1269d328821   Running   k8s-web-to-nginx          default
0f68a2b003811   Running   nginx                     default
aecb3eb0265df   Running   nginx                     default
84d322118a836   Running   k8s-web-to-nginx          default
7557dd00358b7   Running   nginx                     default
7491887c528be   Running   nginx                     default
```

FIGURE 15.7 – Dans le nœud CRI-O, `sudo crictl ps` filtré montre les 3 conteneurs `k8s-web-to-nginx` et les 5 conteneurs `nginx` du namespace `default`, tous exécutés par CRI-O.

Conclusion : Kubernetes ne dépend pas de Docker. Le runtime de conteneurs peut être remplacé (Docker, containerd, CRI-O) sans changer les images ni les fichiers YAML, car tous respectent la CRI et les images respectent le standard OCI.

Nettoyage :

```
minikube delete -p crio
k config use-context minikube
```

```
k get nodes
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ minikube delete -p crio
* Deleting "crio" in docker ...
* Deleting container "crio" ...
* Removing /home/user/.minikube/machines/crio ...
* Removed all traces of the "crio" cluster.
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k config use-context minikube
Switched to context "minikube".
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s$ k get nodes
NAME       STATUS   ROLES                  AGE   VERSION
minikube   Ready    control-plane,master   15h   v1.37.0
```

FIGURE 15.8 – Le cluster `crio` est supprimé et `kubectl` est reconfiguré sur le cluster principal `minikube`, dont le nœud est toujours `Ready`.

Conclusion générale

Le dernier épisode est un résumé, sans manipulation. Au cours de cette partie A, on a :

— créé un cluster local avec Minikube et exploré son nœud et ses composants (`kube-apiserver`, `etcd`, `scheduler`, `controller-manager`, `kube-proxy`, CoreDNS) ;

— manipulé les objets de base : **Pod**, **Deployment**, **ReplicaSet** et **Service** (ClusterIP, NodePort, LoadBalancer) ;

— construit des images personnalisées (Node.js/Express), publiées sur Docker Hub sous `hanine169/...` ;

— réalisé une mise à jour progressive, un rollback, et observé l'auto-réparation des pods ;

— utilisé le tableau de bord Kubernetes ;

— décrit l'infrastructure de façon déclarative avec des fichiers YAML ;

— fait communiquer deux déploiements par le DNS interne (nom de service) ;

— changé le runtime de conteneurs (containerd → CRI-O) sans modifier les applications.